

字符串

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iii
第一章 前缀函数与 KMP 算法	1
1.1 前缀函数	1
1.2 KMP 匹配	1
1.3 顺序同构匹配	3
第二章 Z 函数 (扩展 KMP)	5
2.1 Z 函数	5
2.2 两串扩展 KMP	6
第三章 AC 自动机	7
3.1 AC 自动机出现次数	7
3.2 AC 自动机最短后缀匹配长度	13
3.3 AC 自动机带权匹配	15
3.4 动态 AC: 二进制分组重建	17
第四章 后缀数组 (SA)	19
4.1 后缀数组简介	19
4.1.1 线性后缀数组 (SA-IS)	19
4.1.2 倍增后缀数组与 LCP	23
4.1.3 后缀 LCP 与子串比较	26
4.1.4 两个前缀的最长公共后缀	29
4.1.5 重复子串 AA 的起止位置计数	29
4.1.6 后缀数组: DC3 线性构造	31
第五章 后缀自动机 (SAM)	36
5.1 后缀自动机	36
5.2 SAM 字典序子串查询	41
5.3 SAM 最长公共子串	44
第六章 广义后缀自动机	46
6.1 广义后缀自动机: Trie BFS 构建	46
6.2 指定状态在线扩展 SAM	53
6.3 SAM 文档频率与阈值后缀	56
第七章 Manacher	61
7.1 Manacher 回文半径	61
第八章 回文树	64
8.1 回文树 PAM: 回文后缀与出现次数	64
第九章 最小表示法	67
9.1 最小循环表示	67

第十章 配套知识与验证范围	69
10.1 回文半径与在线回文计数	69
10.2 整数后缀数组的边界与 SAM 次数	69
10.3 广义 SAM 点数与后缀语言	69
 第一部分 跨类依赖	 71
附录 A 数论	72
A.1 模算术简介	72
A.1.1 静态模整数	72
A.2 卢卡斯定理	74
A.2.1 Lucas: 小素数模的大下标组合数	74
附录 B 组合数学	76
B.1 排列组合	76
B.1.1 阶乘组合数	76
附录 C 树状数组	78
C.1 树状数组第 k 小与组合依赖	78

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 前缀函数与 KMP 算法

1.1 前缀函数

`prefix_function(s)` 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
1 vector<int> prefix_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = p[i - 1];
8         while (j && s[i] != s[j]) j = p[j - 1];
9         p[i] = j + (s[i] == s[j]);
10    }
11    return p;
12 }
```

联合使用示例见第 2 页 (Luogu P3375)，其中直接调用本模板。

1.2 KMP 匹配

代码依赖：[prefix_function](#) (第 1 页, 1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`kmp_match(s,t)` 返回非空模式串 t 在 s 中的所有起点 (0-based)，允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
1 vector<int> kmp_match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = prefix_function(t);
5     vector<int> ans;
6     for (int i = 0, j = 0; i < (int)s.size(); i++)
7     {
8         while (j && s[i] != t[j]) j = p[j - 1];
9         j += (s[i] == t[j]);
10        if (j == (int)t.size())
11        {
12            ans.push_back(i - j + 1);
13            j = p[j - 1];
14        }
15    }
16    return ans;
17 }
```

模板题使用：Luogu P3375

题意：求模式串 t 在文本 s 中所有出现的位置，包含重叠匹配；模板返回 0-based 位置，输出加 1。最后一行直接输出 `prefix_function(t)`，它是模式串各前缀的最长真 border 长度。题目输入为非空大写串，函数按字符相等比较，不限于小写。

所需模板：[prefix_function](#) (第 1 页, 1.1 节)、[kmp_match](#) (第 1 页, 1.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3375>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s, t;
6     cin >> s >> t;
7     for (int p : kmp_match(s, t)) cout << p + 1 << '\n';
8     for (int x : prefix_function(t)) cout << x << ' ';
9     cout << '\n';
10 }
```

相关应用：大小和相等关系的顺序同构匹配；KMP 推广。

1.3 顺序同构匹配

`order_match(s,t)` 接收 `vector<long long>`, `t` 非空；返回所有 0-based 起点，含重叠。匹配要求任意两位置的大小及相等关系一致，数值本身可不同。支持完整 signed64，仅比较不作差；输入不变，长度均 $< \text{INT_MAX}$ 。模式预处理记录之前的相等、前驱和后继位置，再用相对下标检查 KMP 扩展。 $O(|t|\log(|t|+1)+|s|)$ 时间， $O(|t| + \text{答案数})$ 空间。P6080 为应用题，不作为正式模板题覆盖。

```

1 // Match all pairwise <, =, > relations; t must be nonempty.
2 vector<int> order_match(const vector<long long> &s, const vector<long long> &t)
3 {
4     assert(!t.empty());
5     int n = s.size(), m = t.size();
6     vector<int> eq(m, -1), lo(m, -1), hi(m, -1), p(m), ans;
7     map<long long, int> last;
8     for (int j = 0; j < m; j++)
9     {
10         auto it = last.lower_bound(t[j]);
11         if (it != last.end() && it->first == t[j])
12             eq[j] = it->second;
13         else
14         {
15             if (it != last.end()) hi[j] = it->second;
16             if (it != last.begin()) lo[j] = prev(it)->second;
17         }
18         last[t[j]] = j;
19     }
20     auto fits = [&](const vector<long long> &a, int i, int j)
21     {
22         int start = i - j;
23         if (eq[j] != -1) return a[i] == a[start + eq[j]];
24         if (lo[j] != -1 && a[i] <= a[start + lo[j]]) return false;
25         if (hi[j] != -1 && a[i] >= a[start + hi[j]]) return false;
26         return true;
27     };
28     for (int i = 1, j = 0; i < m; i++)
29     {
30         while (j && !fits(t, i, j)) j = p[j - 1];
31         if (fits(t, i, j)) j++;
32         p[i] = j;
33     }
34     for (int i = 0, j = 0; i < n; i++)
35     {
36         while (j && !fits(s, i, j)) j = p[j - 1];
37         if (fits(s, i, j)) j++;
38         if (j == m)
39         {
40             ans.push_back(i - m + 1);
41             j = p[j - 1];
42         }
43     }
44     return ans;
45 }

```

应用题补充：Luogu P6080 / Cow Patterns

题意： $N \leq 100000$ 、 $K \leq 25000$ 、 $S \leq 25$ ，值在 $1..S$ ；输出匹配的个数与升序 1-based 起点，逐行输出。保持相等和严格大小关系，允许重叠； S 无需用于算法。应用补充，未在线 AC。

所需模板：`order_match`（第 3 页，1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P6080>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, alphabet;
6     cin >> n >> m >> alphabet;
7     vector<long long> s(n), t(m);
8     for (auto &x : s) cin >> x;
9     for (auto &x : t) cin >> x;
10    auto ans = order_match(s, t);
11    cout << ans.size() << '\n';
12    for (int x : ans) cout << x + 1 << '\n';
13 }
```

第二章 Z 函数 (扩展 KMP)

2.1 Z 函数

`z_function(s)` 返回后缀 `s[i..]` 与原串的 LCP, 约定 `z[0]=n`, 空串返回空数组。O(n)。

```
1 vector<int> z_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if (n) a[0] = n;
6     for (int i = 1, l = 0, r = 0; i < n; i++)
7     {
8         if (i < r) a[i] = min(r - i, a[i - l]);
9         while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
10        if (i + a[i] > r)
11        {
12            l = i;
13            r = i + a[i];
14        }
15    }
16    return a;
17 }
```

模板题使用: Library Checker zalgorithm

题意: 给非空字符串, 对每个起点求其后缀与原串的最长公共前缀长度。直接输出 Z 数组, `z[0]` 等于串长。

所需模板: `z_function` (第 5 页, 2.1 节)。以下仅含使用代码, 默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目: <https://judge.yosupo.jp/problem/zalgorithm>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     auto z = z_function(s);
8     for (int i = 0; i < (int)z.size(); i++)
9         cout << z[i] << (i + 1 == (int)z.size() ? '\n' : ' ');
10 }
```

2.2 两串扩展 KMP

代码依赖：[z_function](#) (第 5 页, 2.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`exkmp(s,t)` 返回 $\{z,p\}$: z 为模式 t 的 Z 数组, $z[0]=|t|$; $p[i]$ 是文本后缀 $s[i..]$ 与 t 的 LCP 长度, 均 0-based。空串合法, t 空时 p 全 0, s 空时 p 为空。任意字节相等比较, 含零字节与高位字节; 不拼接分隔符、不拷贝输入。需要 `z_function`。两串长度均须 $< \text{INT_MAX}$; $O(|s|+|t|)$ 时间, 两个输出 vector 共 $O(|s|+|t|)$ 空间, 返回时移动缓冲区。维护右端最远的匹配区间 $[l,r)$, 区间内利用模式自身 Z 值再向右扩展。P5410 的 1-based 权值乘法应先提升到 long long。

```
1 // Return {Z(t), LCP(s[i..], t) for every i}; empty strings are allowed.
2 pair<vector<int>, vector<int>> exkmp(const string &s, const string &t)
3 {
4     int n = s.size(), m = t.size();
5     auto z = z_function(t);
6     vector<int> p(n);
7     for (int i = 0, l = 0, r = 0; i < n; i++)
8     {
9         if (i < r) p[i] = min(r - i, z[i - l]);
10        while (p[i] < m && p[i] < n - i && s[i + p[i]] == t[p[i]])
11            p[i]++;
12        if (i + p[i] > r)
13        {
14            l = i;
15            r = i + p[i];
16        }
17    }
18    return {move(z), move(p)};
19 }
```

模板题使用：Luogu P5410 / 扩展 KMP

题意: 输入小写串 a, b , 长度均 $1..20000000$ 。求 b 的 Z 数组与 b 对 a 各后缀的 LCP 数组, 各输出 $\text{xor}((i+1)*(长度+1))$ 。 $z[0]=|b|$; 乘法先提升 long long。直接求两串 LCP, 不拼接整串。

所需模板: [z_function](#) (第 5 页, 2.1 节)、`exkmp` (第 6 页, 2.2 节)。以下仅含使用代码, 默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目: <https://www.luogu.com.cn/problem/P5410>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string a, b;
6     cin >> a >> b;
7     auto [z, p] = exkmp(a, b);
8     long long x = 0, y = 0;
9     for (int i = 0; i < (int)z.size(); i++)
10        x ^= 1LL * (i + 1) * (z[i] + 1);
11     for (int i = 0; i < (int)p.size(); i++)
12        y ^= 1LL * (i + 1) * (p[i] + 1);
13     cout << x << '\n' << y << '\n';
14 }
```

第三章 AC 自动机

3.1 AC 自动机出现次数

add(s)/count(s) 默认小写 string; add(s,offset)/count(s,offset) 接受 string 或 vector<int>, 每个元素减 offset 须在 0..25, 同一自动机使用一致编码。字符串字面量请先转 string 再传双参数入口。add 返回终态, 支持空模式返回根 0; count 返回逐状态重叠出现次数, 根恒为文本长度 +1。全部插入后 build 一次, 随后只读且可重复查询; 重复模式共享状态。a[u].len 为 Trie 深度, go/fail 与 order 可读, order 不含根。size 用 a.size(), 重置赋新对象。构建 $O(26V+L)$, 查询 $O(|text|+V)$, 空间 $O(26V)$ 。状态数、深度须在 int 内。非空旧用法不变; 旧根计数语义需减 1。出现过的模式编号数仍按每个终点分别判断 $cnt>0$ 。

```
1 struct AhoCorasick
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0, len = 0;
7     };
8
9     vector<Node> a{Node{}};
10    vector<int> order;
11    bool built = false;
12
13    template <class S>
14    int add(const S &s, int offset)
15    {
16        assert(!built);
17        int u = 0;
18        for (auto ch : s)
19        {
20            long long c = (long long)ch - offset;
21            assert(0 <= c && c < 26);
22            if (!a[u].go[c])
23            {
24                int v = (int)a.size();
25                a[u].go[c] = v;
26                a.push_back(Node{});
27                a[v].len = a[u].len + 1;
28            }
29            u = a[u].go[c];
30        }
31        return u;
32    }
33
34    int add(const string &s)
35    {
36        return add(s, 'a');
37    }
```

构建失败链接与统计出现次数（接上页同一结构体）：

```

39 void build()
40 {
41     assert(!built);
42     built = true;
43     queue<int> q;
44     for (int v : a[0].go)
45         if (v) q.push(v);
46     while (!q.empty())
47     {
48         int u = q.front();
49         q.pop();
50         order.push_back(u);
51         for (int c = 0; c < 26; c++)
52         {
53             int v = a[u].go[c];
54             if (v)
55             {
56                 a[v].fail = a[a[u].fail].go[c];
57                 q.push(v);
58             }
59             else
60                 a[u].go[c] = a[a[u].fail].go[c];
61         }
62     }
63 }
64
65 template <class S>
66 vector<long long> count(const S &s, int offset) const
67 {
68     assert(built);
69     vector<long long> ans(a.size());
70     ans[0] = 1;
71     int u = 0;
72     for (auto ch : s)
73     {
74         long long c = (long long)ch - offset;
75         assert(0 <= c && c < 26);
76         u = a[u].go[c];
77         ++ans[u];
78     }
79     for (int i = (int)order.size() - 1; i >= 0; i--)
80         ans[a[order[i]].fail] += ans[order[i]];
81     return ans;
82 }
83
84 vector<long long> count(const string &s) const
85 {
86     return count(s, 'a');
87 }
88 };

```

模板题使用：Luogu P5357

题意：给多个小写字母模式和文本，输出每个模式的出现次数，允许重叠。先保存 add 返回的终止状态，全部插入后 build，再 count 文本并按状态取值；重复模式共享状态，仍按输入顺序输出。

所需模板：[AhoCorasick](#) (第 7 页, 3.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5357>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     AhoCorasick ac;
8     vector<int> terminal(n);
9     for (int &id : terminal)
10    {
11        string pattern;
12        cin >> pattern;
13        id = ac.add(pattern);
14    }
15    ac.build();
16    string text;
17    cin >> text;
18    auto count = ac.count(text);
19    for (int id : terminal) cout << count[id] << '\n';
20 }
```

模板题使用：Luogu P3808 / AC 自动机简单版

题意：给非空小写模式串和文本，求出现过的模式编号数；两个相同字符串但编号不同要分别计数，同一编号多次出现只算一次。模式数、模式总长和文本长度均至 1000000。保存每次 add 返回的终点，count 后逐编号判断是否为正；不清空终态，因此可重复查询。

所需模板：[AhoCorasick](#)（第 7 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3808>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     AhoCorasick ac;
8     vector<int> end(n);
9     for (int &u : end)
10    {
11        string s;
12        cin >> s;
13        u = ac.add(s);
14    }
15    ac.build();
16    string t;
17    cin >> t;
18    auto cnt = ac.count(t);
19    int ans = 0;
20    for (int u : end) ans += cnt[u] > 0;
21    cout << ans << '\n';
22 }
```

接口用法演示：整数序列 AC 与深度（接口演示）

演示约定：输入 n, q, offset ，每条模式/查询给出长度和整数序列。各元素减 offset 必须为 $0..25$ ，允许空序列。先输出各模式 Trie 深度，再逐查询输出各模式出现次数；空模式贡献文本长度 $+1$ 。

所需模板：[AhoCorasick](#)（第 7 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://oi-wiki.org/string/ac-automaton/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q, offset;
6     cin >> n >> q >> offset;
7     AhoCorasick ac;
8     vector<int> end(n);
9     for (int &u : end)
10    {
11        int m;
12        cin >> m;
13        vector<int> s(m);
14        for (int &x : s) cin >> x;
15        u = ac.add(s, offset);
16    }
17    ac.build();
18    for (int u : end) cout << ac.a[u].len << ' ';
19    cout << '\n';
20    while (q--)
21    {
22        int m;
23        cin >> m;
24        vector<int> s(m);
25        for (int &x : s) cin >> x;
26        auto cnt = ac.count(s, offset);
27        for (int u : end) cout << cnt[u] << ' ';
28        cout << '\n';
29    }
30 }
```

接口用法演示：自定义字符偏移 AC（接口演示）

演示约定：输入 n, q 及字母表起始字符 $offset$ ，再输入 n 个模式串和 q 个文本，均用双引号包围以支持空串。字符减 $offset$ 为 $0..25$ 。逐查询输出所有模式的重叠出现次数；例如 $offset$ 为 A 可处理大写字母。

所需模板：[AhoCorasick](#)（第 7 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://oi-wiki.org/string/ac-automaton/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     char offset;
7     cin >> n >> q >> offset;
8     AhoCorasick ac;
9     vector<int> end(n);
10    for (int &u : end)
11    {
12        string s;
13        cin >> quoted(s);
14        u = ac.add(s, offset);
15    }
16    ac.build();
17    while (q--)
18    {
19        string s;
20        cin >> quoted(s);
21        auto cnt = ac.count(s, offset);
22        for (int u : end) cout << cnt[u] << ' ';
23        cout << '\n';
24    }
25 }
```

3.2 AC 自动机最短后缀匹配长度

代码依赖：[AhoCorasick](#)（第 7 页，3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

先用 `AhoCorasick.add(s)` 得到终止节点，记录 (节点, 模式长度)，再 `build` 后调用 `ac_shortest(ac, ends)`。返回每个状态沿 `fail` 链对应的最短已登记模式长度，无模式为 `INT_MAX`。登记长度须真实且在 $1..INT_MAX-1$ ；重复模式、部分模式集合、空集合均可，未登记的模式不会参与最小值。按 `ac.order` 的 BFS 顺序传播，父 `fail` 状态先完成； $O(\text{状态数} + \text{登记数})$ 时间， $O(\text{状态数})$ 额外空间。扫描文本时 `u=ac.a[u].go[c]`，`len[u]` 就是当前位置结尾的最短匹配长度；文本仍须为小写字母。辅助函数不修改自动机，也不影响 `count(text)`，可对同一自动机的不同登记集合重复调用。

```

1 vector<int> ac_shortest(const AhoCorasick &ac, const vector<pair<int, int>> &ends)
2 {
3     assert(ac.built);
4     vector<int> len(ac.a.size(), INT_MAX);
5     for (auto [u, n] : ends)
6     {
7         assert(0 < u && u < (int)len.size() && 0 < n && n < INT_MAX);
8         len[u] = min(len[u], n);
9     }
10    for (int u : ac.order) len[u] = min(len[u], len[ac.a[u].fail]);
11    return len;
12 }
```

应用题补充：AC 最短后缀匹配（本地接口示例）

题意：输入模式数、各个非空小写模式串及文本，逐位置输出以该位置结尾的最短模式长度，无匹配输出 0。先登记终止节点与长度，再 build 和传播。示例对应附件 min_len 变体，只作本地接口演示，不是 OJ 提交记录。

所需模板：[AhoCorasick](#)（第 7 页，3.1 节）、[ac_shortest](#)（第 13 页，3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://oi-wiki.org/string/ac-automaton/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     AhoCorasick ac;
8     vector<pair<int, int>> ends;
9     while (n--)
10     {
11         string s;
12         cin >> s;
13         ends.push_back({ac.add(s), (int)s.size()});
14     }
15     ac.build();
16     auto len = ac_shortest(ac, ends);
17     string s;
18     cin >> s;
19     int u = 0;
20     for (char c : s)
21     {
22         u = ac.a[u].go[c - 'a'];
23         cout << (len[u] == INT_MAX ? 0 : len[u]) << ' ';
24     }
25     cout << '\n';
26 }
```

3.3 AC 自动机带权匹配

代码依赖：[AhoCorasick](#) (第 7 页, 3.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

构造接收 (小写模式串, long long 权重) 列表；重复串权重相加，可负、可零。sum[u] 为当前状态所有后缀模式的权重和，query(text) 累加全部重叠匹配， $O(|\text{text}|)$ 。允许空串，在每个边界匹配，共 $|\text{text}|+1$ 次；空模式集合返回 0。构造 $O(26V+L+n)$ ，空间 $O(26V+n)$ ，L 为总模式长。对象自持有 ac 和 sum，可复制、移动；构造后成员只读，重置赋新对象。所有中间和及答案须在 long long 内。

```

1 struct ACWeighted
2 {
3     AhoCorasick ac;
4     vector<long long> sum;
5
6     ACWeighted(const vector<pair<string, long long>> &patterns = {})
7     {
8         vector<pair<int, long long>> ends;
9         for (const auto &[s, w] : patterns)
10         {
11             int u = s.empty() ? 0 : ac.add(s);
12             ends.push_back({u, w});
13         }
14         ac.build();
15         sum.resize(ac.a.size());
16         for (auto [u, w] : ends) sum[u] += w;
17         for (int u : ac.order) sum[u] += sum[ac.a[u].fail];
18     }
19
20     long long query(const string &s) const
21     {
22         long long ans = sum[0];
23         int u = 0;
24         for (char c : s)
25         {
26             assert('a' <= c && c <= 'z');
27             u = ac.a[u].go[c - 'a'];
28             ans += sum[u];
29         }
30         return ans;
31     }
32 };

```

接口用法演示：静态带权多模式查询（接口演示）

演示约定：输入 n, q ，随后 n 组带引号模式串及有符号权重，再读 q 个带引号文本。双引号允许表示空串。输出全部重叠匹配的权重和；空模式在每个边界匹配。

所需模板：[AhoCorasick](#)（第 7 页，3.1 节）、[ACWeighted](#)（第 15 页，3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://oi-wiki.org/string/ac-automaton/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     vector<pair<string, long long>> patterns(n);
8     for (auto &[s, w] : patterns) cin >> quoted(s) >> w;
9     ACWeighted ac(patterns);
10    while (q--)
11    {
12        string s;
13        cin >> quoted(s);
14        cout << ac.query(s) << '\n';
15    }
16 }
```

3.4 动态 AC：二进制分组重建

代码依赖：ACWeighted（第 15 页，3.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

add(s,w) 累加模式权重；插入计数用 +1，删除用-1，合法删除由调用者保证。query(s) 求所有模式出现次数乘当前权重之和，支持重复、负权和空串，边界同 ACWeighted。第 k 组为空或含 2^k 次更新；进位合并后重建。M 次更新、总长度 L，累计构建 $O(26(L+M)\log(M+2))$ ，查询 $O((|s|+1)\log(M+2))$ ，空间 $O(26(L+M))$ ；一次更新可线性重建，非最坏对数。历史删除不会回收更新记录，空间按全部历史更新计。block/ac 只读；重置赋新对象。所有组内及跨组中间和须在 long long 内。

```

1 struct DynamicAC
2 {
3     vector<vector<pair<string, long long>>> block;
4     vector<ACWeighted> ac;
5
6     void add(string s, long long w)
7     {
8         vector<pair<string, long long>> cur;
9         cur.push_back({move(s), w});
10        int k = 0;
11        while (k < (int)block.size() && !block[k].empty())
12        {
13            for (auto &p : block[k]) cur.push_back(move(p));
14            block[k].clear();
15            ac[k] = ACWeighted();
16            k++;
17        }
18        if (k == (int)block.size())
19        {
20            block.emplace_back();
21            ac.emplace_back();
22        }
23        block[k] = move(cur);
24        ac[k] = ACWeighted(block[k]);
25    }
26
27    long long query(const string &s) const
28    {
29        long long ans = 0;
30        for (int k = 0; k < (int)block.size(); k++)
31        {
32            if (!block[k].empty()) ans += ac[k].query(s);
33        }
34        return ans;
35    }
36 };

```

应用题补充：Codeforces 710F / String Set Queries

题意：在线维护字符串集合。1 插入，2 删除，3 询问文本中所有集合模式的总出现次数。题目保证新增串从未加入、删除串存在；每次回答必须 endl 刷新后再读下一条，不能预读全部操作。用 +1/-1 更新权重。

所需模板：AhoCorasick (第 7 页, 3.1 节)、ACWeighted (第 15 页, 3.3 节)、DynamicAC (第 17 页, 3.4 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://codeforces.com/problemset/problem/710/F>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int m;
6     cin >> m;
7     DynamicAC dict;
8     while (m--)
9     {
10         int t;
11         string s;
12         cin >> t >> s;
13         if (t == 3)
14         {
15             cout << dict.query(s) << endl;
16         }
17         else
18         {
19             dict.add(move(s), t == 1 ? 1 : -1);
20         }
21     }
22 }
```

第四章 后缀数组 (SA)

4.1 后缀数组简介

4.1.1 线性后缀数组 (SA-IS)

SAIS(s,alphabet): 整数符号 $0..alphabet-1$, $1 \leq alphabet < INT_MAX$, 长度 $n < INT_MAX$; SAIS(string) 按 unsigned byte 处理, 支持零字节。sa/rk/lcp 均长 n , 0-based; lcp[0]=0, 其余为相邻后缀 LCP。无需哨兵, 空串结果为空。诱导排序时间/空间 $O(n+alphabet)$, 递归深度 $O(\log n)$; 任意值离散化另需 $O(n \log n)$ 。保留倍增 SuffixArray; SuffixLCP 不直接接受本类型。

```
1 struct SAIS
2 {
3     vector<int> sa, rk, lcp;
4
5     SAIS(const string &s)
6     {
7         assert(s.size() < INT_MAX);
8         vector<int> a;
9         a.reserve(s.size());
10        for (unsigned char c : s) a.push_back(c);
11        *this = SAIS(a, 256);
12    }
13
14    SAIS(const vector<int> &s, int alphabet)
15    {
16        assert(s.size() < INT_MAX && 1 <= alphabet && alphabet < INT_MAX);
17        for (int c : s) assert(0 <= c && c < alphabet);
18        int n = s.size();
19        sa = sort_suffixes(s, alphabet);
20        rk.resize(n);
21        lcp.assign(n, 0);
22        for (int i = 0; i < n; i++) rk[sa[i]] = i;
23        for (int i = 0, k = 0; i < n; i++)
24        {
25            if (!rk[i])
26            {
27                k = 0;
28                continue;
29            }
30            int j = sa[rk[i] - 1];
31            while (k < n - i && k < n - j && s[i + k] == s[j + k]) k++;
32            lcp[rk[i]] = k;
33            if (k) k--;
34        }
35    }
```

类型与桶内诱导（接上页同一结构体）：

```

37 private:
38     static vector<int> sort_suffixes(const vector<int> &s, int alphabet)
39     {
40         int n = s.size();
41         if (!n) return {};
42         if (n == 1) return {0};
43         vector<int> sa(n), lo(alphabet + 1), hi(alphabet + 1);
44         vector<bool> type(n);
45         for (int i = n - 2; i >= 0; i--)
46             type[i] = s[i] == s[i + 1] ? type[i + 1] : s[i] < s[i + 1];
47         for (int i = 0; i < n; i++)
48             if (type[i]) lo[s[i] + 1]++;
49             else hi[s[i]]++;
50         for (int c = 0; c < alphabet; c++)
51         {
52             hi[c] += lo[c];
53             lo[c + 1] += hi[c];
54         }
55         auto induce = [&](const vector<int> &lms)
56         {
57             fill(sa.begin(), sa.end(), -1);
58             vector<int> pos = hi;
59             for (int i : lms) sa[pos[s[i]]++] = i;
60             pos = lo;
61             sa[pos[s[n - 1]]++] = n - 1;
62             for (int i = 0; i < n; i++)
63             {
64                 int v = sa[i];
65                 if (v > 0 && !type[v - 1]) sa[pos[s[v - 1]]++] = v - 1;
66             }
67             pos = lo;
68             for (int i = n - 1; i >= 0; i--)
69             {
70                 int v = sa[i];
71                 if (v > 0 && type[v - 1]) sa[--pos[s[v - 1] + 1]] = v - 1;
72             }
73         };

```

LMS 命名与递归（接上页同一结构体）：

```

74     vector<int> id(n, -1), lms;
75     for (int i = 1; i < n; i++)
76         if (!type[i - 1] && type[i])
77         {
78             id[i] = lms.size();
79             lms.push_back(i);
80         }
81     induce(lms);
82     int m = lms.size();
83     if (m)
84     {
85         vector<int> order, reduced(m);
86         for (int v : sa)
87             if (id[v] >= 0) order.push_back(v);
88         int names = 1;
89         for (int i = 1; i < m; i++)
90         {
91             int a = order[i - 1], b = order[i];
92             int x = id[a] + 1 < m ? lms[id[a] + 1] : n;
93             int y = id[b] + 1 < m ? lms[id[b] + 1] : n;
94             bool same = x - a == y - b;
95             if (same)
96             {
97                 while (a < x && s[a] == s[b]) a++, b++;
98                 same = a < n && b < n && s[a] == s[b];
99             }
100             if (!same) names++;
101             reduced[id[order[i]]] = names - 1;
102         }
103         order = sort_suffixes(reduced, names);
104         for (int &i : order) i = lms[i];
105         induce(order);
106     }
107     return sa;
108 }
109 };

```

模板题使用：Library Checker suffixarray

题意：输入长度 1..500000 的小写字符串，按字典序输出所有非空后缀的 0-based 起点。独立 SA-IS 实现；无需附加哨兵，不继承倍增版本的线上记录。

所需模板：SAIS（第 19 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://judge.yosupo.jp/problem/suffixarray>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     SAIS suffix(s);
8     for (int i = 0; i < (int)s.size(); i++)
9         cout << suffix.sa[i] << (i + 1 == (int)s.size() ? '\n' : ' ');
10 }
```

4.1.2 倍增后缀数组与 LCP

构造 SuffixArray(s) 支持任意字节串, 按 unsigned char 的 0..255 数值比较, 包含内嵌零字节; 也可构造 SuffixArray(a,alphabet), 整数序列要求 $0 \leq a[i] < \text{alphabet}$, $1 \leq \text{alphabet} < \text{INT_MAX}$ 。重新赋值 SuffixArray(a,alphabet) 可重建。长度须小于 INT_MAX。sa[k] 为第 k 个非空后缀起点, rk[i] 为排名, 全部 0-based; lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀, lcp[0]=0。空序列合法, 三数组均为空。无需手动追加哨兵, 也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序, 倍增 $O(n \log n + \text{alphabet})$, 空间 $O(n + \text{alphabet})$; 字节串 alphabet=256。大整数先自行保序离散化, 避免按最大原值开桶。Kasai 求 lcp 为 $O(n)$ 。不同起点 x,y 的 LCP 为 $\text{lcp}[\min(\text{rk}[x], \text{rk}[y]) + 1.. \max(\text{rk}[x], \text{rk}[y])]$ 的最小值; 本结构不内建 RMQ, 需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造, 不替代 DC3/SA-IS。

```

1 struct SuffixArray
2 {
3     vector<int> sa, rk, lcp;
4
5     SuffixArray(const string &s)
6     {
7         vector<int> a;
8         a.reserve(s.size());
9         for (unsigned char c : s) a.push_back(c);
10        *this = SuffixArray(a, 256);
11    }
12
13    SuffixArray(const vector<int> &s, int alphabet)
14    {
15        assert(s.size() < INT_MAX);
16        assert(1 <= alphabet && alphabet < INT_MAX);
17        int n = (int)s.size();
18        sa.resize(n);
19        rk.resize(n);
20        lcp.assign(n, 0);
21        if (!n) return;
22        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
23        int m = alphabet;
24        for (int i = 0; i < n; i++)
25        {
26            assert(0 <= s[i] && s[i] < alphabet);
27            rk[i] = s[i] + 1;
28            cnt[rk[i]]++;
29        }
30        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
31        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;

```

```

32     for (int k = 1; k < n; k *= 2)
33     {
34         int p = 0;
35         for (int i = n - k; i < n; i++) order[p++] = i;
36         for (int i : sa)
37             if (i >= k) order[p++] = i - k;
38         fill(cnt.begin(), cnt.begin() + m + 1, 0);
39         for (int x : rk) cnt[x]++;
40         for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
41         for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
42         old.swap(rk);
43         auto second = [&](int i)
44         {
45             return i < n - k ? old[i + k] : 0;
46         };
47         m = 1;
48         rk[sa[0]] = m;
49         for (int i = 1; i < n; i++)
50         {
51             int a = sa[i - 1], b = sa[i];
52             if (old[a] != old[b] || second(a) != second(b)) m++;
53             rk[b] = m;
54         }
55         if (m == n) break;
56     }
57     for (int i = 0; i < n; i++) rk[sa[i]] = i;
58     for (int i = 0, k = 0; i < n; i++)
59     {
60         if (!rk[i])
61         {
62             k = 0;
63             continue;
64         }
65         int j = sa[rk[i] - 1];
66         while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
67         lcp[rk[i]] = k;
68         if (k) --k;
69     }
70 }
71 };

```

模板题使用：Luogu P3809

题意：把所有非空后缀按字典序排序，输出各后缀在原串中的起点。模板 sa 为 0-based，题目输出为 1-based，因此加 1。

所需模板：SuffixArray (第 23 页, 4.1.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3809>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     std::string s;
6     std::cin >> s;
7     SuffixArray suffix(s);
8     for (int i = 0; i < (int)s.size(); i++)
9         std::cout << suffix.sa[i] + 1 << (i + 1 == (int)s.size() ? '\n' : ' ');
10 }
```

模板题使用：Library Checker longest_common_substring

题意：求两个小写字母串的一段最长公共子串，输出两串各自 0-based 半开区间，各串长度至 50 万。整数拼接以 26 为分隔符、alphabet=27；扫描异来源相邻后缀，并将 LCP 截断至原串边界。

所需模板：SuffixArray (第 23 页, 4.1.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/longest_common_substring

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s, t;
6     cin >> s >> t;
7     int n = s.size(), m = t.size();
8     vector<int> a;
9     for (char c : s) a.push_back(c - 'a');
10    a.push_back(26);
11    for (char c : t) a.push_back(c - 'a');
12    SuffixArray suffix(a, 27);
13    int best = 0, left = 0, right = 0;
14    for (int k = 1; k < (int)a.size(); k++)
15    {
16        int x = suffix.sa[k - 1], y = suffix.sa[k];
17        if (x > y) swap(x, y);
18        if (x >= n || y <= n) continue;
19        int len = min({suffix.lcp[k], n - x, n + 1 + m - y});
20        if (len > best)
21        {
22            best = len;
23            left = x;
24            right = y - n - 1;
25        }
26    }
27    cout << left << ' ' << left + best << ' ' << right << ' ' << right + best << '\n';
28 }
```

4.1.3 后缀 LCP 与子串比较

代码依赖：SuffixArray (第 23 页, 4.1.2 节)。抄写时一并准备；标准库类型不列入算法依赖。

先构造 SuffixArray, 再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据, 不保存原字符串或源对象引用, 源对象可销毁或重建。query(x,y) 返回 $s[x..n]$ 与 $s[y..n]$ 的最长公共前缀, $0 \leq x, y \leq n$, 允许空后缀; 同起点返回 $n-x$ 。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串, 允许空串, 返回 -1/0/1。若公共前缀覆盖较短子串, 按长度比较, 否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表 $O(n \log n)$ 时间、空间, 每次查询 $O(1)$, 无需哈希和原串。 n 、 rk 、 lg 、 st 可读, 禁止调用方破坏。

```

1 struct SuffixLCP
2 {
3     int n;
4     vector<int> rk, lg;
5     vector<vector<int>> st;
6
7     SuffixLCP(const SuffixArray &suffix)
8     {
9         n = suffix.sa.size();
10        rk = suffix.rk;
11        lg.assign(n + 1, 0);
12        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
13        st.assign(lg[n] + 1, {});
14        st[0] = suffix.lcp;
15        for (int k = 1; k <= lg[n]; k++)
16        {
17            int len = 1 << k;
18            st[k].resize(n - len + 1);
19            for (int i = 0; i <= n - len; i++)
20                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
21        }
22    }
23
24    int query(int x, int y) const
25    {
26        assert(0 <= x && x <= n && 0 <= y && y <= n);
27        if (x == y || x == n || y == n) return n - max(x, y);
28        int a = rk[x], b = rk[y];
29        if (a > b) swap(a, b);
30        int k = lg[b - a];
31        return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
32    }
33
34    // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
35    int compare(int l1, int r1, int l2, int r2) const
36    {
37        assert(0 <= l1 && l1 <= r1 && r1 <= n);
38        assert(0 <= l2 && l2 <= r2 && r2 <= n);
39        int a = r1 - l1, b = r2 - l2;
40        if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
41        return rk[l1] < rk[l2] ? -1 : 1;
42    }
43 };

```

接口用法演示：静态后缀查询（自定义协议）

演示约定：自定义接口演示，非官方题目或 OJ 驱动。首行 n q ，次行恰为 n 个小写字母（ $n=0$ 时保留空行）； $0 \leq n, q \leq 200000$ 仅为演示上限。操作 $0\ x\ y$ 输出 $s[x..n)$ 、 $s[y..n)$ 的 LCP； $1\ l1\ r1\ l2\ r2$ 比较两个半开子串，输出 $-1/0/1$ ； $2\ x\ y$ 输出前缀 $s[0..x)$ 、 $s[0..y)$ 的最长公共后缀长度，LCS 在此不是子序列。所有端点 $0..n$ ，允许空区间；同一起点 LCP 为 $n-x$ ，同一前缀 LCS 为 x 。反向结构必须从原串倒序构造。子串 LCP 取 $\min(\text{query}(l1, l2), r1-l1, r2-l2)$ ，子串公共后缀取 $\min(\text{prefix_lcs}(\text{reversed}, r1, r2), r1-l1, r2-l2)$ 。预处理 $O(n \log n)$ 时间/空间，各查询 $O(1)$ ；字符串修改后重建。

所需模板：SuffixArray（第 23 页，4.1.2 节）、SuffixLCP（第 26 页，4.1.3 节）、prefix_lcs（第 29 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCTemplate/blob/main/src/compact/suffix_lcp.hpp

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     if (!(cin >> n >> q)) return 0;
7     assert(0 <= n && n <= 200000 && 0 <= q && q <= 200000);
8     cin.ignore(numeric_limits<streamsize>::max(), '\n');
9     string s;
10    getline(cin, s); // Keep the empty line when n = 0; do not use cin >> ws.
11    assert((int)s.size() == n);
12    for (char c : s) assert('a' <= c && c <= 'z');
13    SuffixLCP forward{SuffixArray(s)};
14    string r(s.rbegin(), s.rend());
15    SuffixLCP reversed{SuffixArray(r)};
16    while (q--)
17    {
18        int op;
19        cin >> op;
20        if (op == 0)
21        {
22            int x, y;
23            cin >> x >> y;
24            cout << forward.query(x, y) << '\n';
25        }
26        else if (op == 1)
27        {
28            int l1, r1, l2, r2;
29            cin >> l1 >> r1 >> l2 >> r2;
30            cout << forward.compare(l1, r1, l2, r2) << '\n';
31        }
32        else
33        {
34            assert(op == 2);
35            int x, y;
36            cin >> x >> y;
37            cout << prefix_lcs(reversed, x, y) << '\n';
38        }
39    }
40 }
```

接口用法演示：kuangbin DA 来源模型：反串与 LCP 求最长回文

演示约定：来源程序协议：读入非空字符串直到 EOF，每组输出最靠左的最长回文子串。组合原串、独立分隔符与反串，通过奇偶中心的 LCP 求半径； $i=0$ 的偶中心查询空后缀，返回 0。新实现按 unsigned char 加 2，分隔符为 1；长度须 $\leq (\text{INT_MAX}-2)/2$ 且满足内存限制。O($n \log n$) 时间空间。此为来源应用协议演示，不是独立线上模板题或 AC；仅需最长回文时优先用 Manacher。

所需模板：SuffixArray（第 23 页，4.1.2 节）、SuffixLCP（第 26 页，4.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://github.com/kuangbin/ACM-ICPC>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     while (cin >> s)
7     {
8         assert(s.size() <= (INT_MAX - 2) / 2);
9         int len = s.size();
10        int n = 2 * len + 1;
11        vector<int> a(n);
12        for (int i = 0; i < len; i++)
13        {
14            a[i] = (unsigned char)s[i] + 2;
15            a[len + 1 + i] = (unsigned char)s[len - 1 - i] + 2;
16        }
17        a[len] = 1;
18        SuffixArray suffix(a, 258);
19        SuffixLCP lcp(suffix);
20        int best = 0, start = 0;
21        for (int i = 0; i < len; i++)
22        {
23            int radius = lcp.query(i, n - i);
24            if (2 * radius > best)
25            {
26                best = 2 * radius;
27                start = i - radius;
28            }
29            radius = lcp.query(i, n - i - 1);
30            if (2 * radius - 1 > best)
31            {
32                best = 2 * radius - 1;
33                start = i - radius + 1;
34            }
35        }
36        cout << s.substr(start, best) << '\n';
37    }
38    return 0;
39 }

```

4.1.4 两个前缀的最长公共后缀

代码依赖: [SuffixLCP](#) (第 26 页, 4.1.3 节)。抄写时一并准备; 标准库类型不列入算法依赖。

`prefix_lcs(reversed,x,y)` 返回原序列 `s[0..x)` 与 `s[0..y)` 的最长公共后缀, 端点满足 $0 \leq x, y \leq n$ 。reversed 必须在原序列的反转序列上构造 SuffixLCP, 不能传正向对象; 转换为 `reversed.query(n-x,n-y)`, 单次 $O(1)$ 。只需前向 LCP 时不必构造反向对象; 该接口中的 LCS 是最长公共后缀, 不是最长公共子序列。对于两个任意子串, 以右端点查询后还需截断到两个子串长度的最小值。

```
1 // reversed must be built on the reversed original sequence.
2 int prefix_lcs(const SuffixLCP &reversed, int x, int y)
3 {
4     assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
5     return reversed.query(reversed.n - x, reversed.n - y);
6 }
```

联合使用示例见第 27 页 (静态后缀查询 (自定义协议)), 其中直接调用本模板。

4.1.5 重复子串 AA 的起止位置计数

代码依赖: [SuffixLCP](#) (第 26 页, 4.1.3 节)、[prefix_lcs](#) (第 29 页, 4.1.4 节)。抄写时一并准备; 标准库类型不列入算法依赖。

`square_counts(forward,reversed)` 返回 `(start,finish)` 两个长度 `n` 的 long long 数组, `start[i]/finish[i]` 为开始/结束于位置 `i` 的非空 AA 型子串个数; 结束位置是最后一个字符下标, 均 0-based。两对象必须分别对应同一原序列及其反序列, 长度相同, 允许空序列。枚举半长 `len`, 并以其整数倍 `i` 为锚点; 向左匹配截断为 `len-1`, 向右截断为 `len`, 由 LCP/LCS 得出合法起点连续区间, 差分累计。左侧不能放宽到 `len`, 否则相邻锚点会重复计数。预处理查询器后, 此函数 $O(n \log n)$ 时间、 $O(n)$ 额外空间。不同出现位置、不同半长分别计数; 不要求 A 为最小周期。P1117 的 AABB 拆分数为所有分界点 `finish[i]*start[i+1]` 之和, 累加类型需按题目范围选择, 函数本身只返回起止计数。

```
1 // Count nonempty squares AA by their starting and ending positions.
2 pair<vector<long long>, vector<long long>> square_counts(const SuffixLCP &forward,
3                                                         const SuffixLCP &reversed)
4 {
5     int n = forward.n;
6     assert(reversed.n == n);
7     vector<long long> start(n + 1), finish(n + 1);
8     for (int len = 1; len <= n / 2; len++)
9         for (int i = 0; i < n - len; i += len)
10             {
11                 int j = i + len;
12                 int left = min(len - 1, prefix_lcs(reversed, i, j));
13                 int right = min(len, forward.query(i, j));
14                 if (left + right < len) continue;
15                 int l = i - left, r = i + right - len;
16                 start[l]++;
17                 start[r + 1]--;
18                 finish[l + 2 * len - 1]++;
19                 finish[r + 2 * len]--;
20             }
```

```

21     for (int i = 1; i < n; i++)
22     {
23         start[i] += start[i - 1];
24         finish[i] += finish[i - 1];
25     }
26     start.resize(n);
27     finish.resize(n);
28     return {start, finish};
29 }

```

应用题补充：Luogu P1117 [NOI2016] 优秀的拆分

题意：统计所有子串的非空 AABB 拆分，不同出现位置、不同 A/B 长度分别计数，允许 A=B。T 为 1 至 10，每组小写串长度不超过 30000。正向与反向 SuffixLCP 必须来自同一字符串；start[i] 和 finish[i] 分别计开始/结束于字符 i 的 AA。每个分界点贡献 finish[i]*start[i+1]，以 long long 累加。此为 NOI 比赛应用题，尚未计入独立模板题覆盖。

所需模板：SuffixArray (第 23 页, 4.1.2 节)、SuffixLCP (第 26 页, 4.1.3 节)、prefix_lcs (第 29 页, 4.1.4 节)、square_counts (第 29 页, 4.1.5 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P1117>

```

1  int main()
2  {
3      std::ios::sync_with_stdio(false);
4      std::cin.tie(nullptr);
5      int tests;
6      std::cin >> tests;
7      while (tests--)
8      {
9          std::string s;
10         std::cin >> s;
11         int n = s.size();
12         SuffixLCP forward{SuffixArray(s)};
13         std::reverse(s.begin(), s.end());
14         SuffixLCP backward{SuffixArray(s)};
15         auto [start, finish] = square_counts(forward, backward);
16         long long answer = 0;
17         for (int i = 0; i + 1 < n; i++) answer += finish[i] * start[i + 1];
18         std::cout << answer << '\n';
19     }
20 }

```

4.1.6 后缀数组：DC3 线性构造

DC3(string) 按 unsigned char 比较全部字节 (含 0/255); DC3(vector<int>,alphabet) 要求每项在 $[0, \text{alphabet})$, $1 \leq \text{alphabet} < \text{INT_MAX} - 3$, 长度 $n < \text{INT_MAX} - 3$ 。空输入合法。sa 为所有非空后缀的 0-based 起点, $\text{rk}[\text{sa}[i]] = i$; $\text{lcp}[0] = 0$, $\text{lcp}[i]$ 为 $\text{sa}[i-1]$ 与 $\text{sa}[i]$ 的公共前缀长度。三数组均长 n , 空输入均为空, 不含空后缀; 输入不被改写, 无需调用者补零或三倍数组。内部把符号加一, 0 只作哨兵; $n \bmod 3 = 1$ 时补一个虚拟采样点, 归并时跳过。稳定基数排序三元组、递归命名、排序余 0 类并合并; 时间空间均 $O(n + \text{alphabet})$, 仅 $\text{alphabet} = O(n)$ 或固定字节集时称线性。稀疏大整数先保序离散化, 其比较排序代价另计。递归深度 $O(\log(n+1))$, 私有 `sort_suffixes` 要求正符号, 用户只用构造及结果数组。与 `SuffixArray/SAIS` 接口约定一致, 无线上性能优越性声明。

```

1 struct DC3
2 {
3     vector<int> sa, rk, lcp;
4
5     DC3(const string &s)
6     {
7         vector<int> a;
8         for (unsigned char c : s)
9             a.push_back(c);
10        *this = DC3(a, 256);
11    }
12
13    DC3(const vector<int> &s, int alphabet)
14    {
15        assert(s.size() < INT_MAX - 3 && 1 <= alphabet && alphabet < INT_MAX - 3);
16        int n = s.size();
17        vector<int> a(n);
18        for (int i = 0; i < n; i++)
19        {
20            assert(0 <= s[i] && s[i] < alphabet);
21            a[i] = s[i] + 1;
22        }
23        sa = sort_suffixes(move(a), alphabet);
24        rk.resize(n);
25        lcp.assign(n, 0);
26        for (int i = 0; i < n; i++)
27            rk[sa[i]] = i;
28        for (int i = 0, k = 0; i < n; i++)
29        {
30            if (!rk[i])
31            {
32                k = 0;
33                continue;
34            }
35            int j = sa[rk[i] - 1];
36            while (k < n - i && k < n - j && s[i + k] == s[j + k])
37                k++;
38            lcp[rk[i]] = k;
39            if (k)
40                k--;
41        }
42    }

```

采样三元组排序与递归命名（接上页同一结构体）：

```

44 private:
45     static vector<int> sort_suffixes(vector<int> s, int alphabet)
46     {
47         int n = s.size();
48         if (!n)
49             return {};
50         if (n == 1)
51             return {0};
52         s.resize(n + 3);
53         int n0 = (n + 2) / 3;
54         int extra = n % 3 == 1;
55         vector<int> sample, zero;
56         for (int i = 0; i < n + extra; i++)
57             if (i % 3)
58                 sample.push_back(i);
59         auto radix = [&](vector<int> &a, int shift)
60         {
61             vector<int> count(alphabet + 1, b(a.size()));
62             for (int i : a)
63                 count[s[i + shift]]++;
64             for (int i = 1; i <= alphabet; i++)
65                 count[i] += count[i - 1];
66             for (int i = (int)a.size() - 1; i >= 0; i--)
67                 b[--count[s[a[i] + shift]]] = a[i];
68             a.swap(b);
69         };
70         for (int shift = 2; shift >= 0; shift--)
71             radix(sample, shift);
72         int names = 0;
73         vector<int> reduced(sample.size(), order(sample.size()));
74         array<int, 3> last{-1, -1, -1};
75         for (int i : sample)
76         {
77             array<int, 3> key{s[i], s[i + 1], s[i + 2]};
78             if (key != last)
79                 names++;
80             last = key;
81             reduced[i / 3 + (i % 3 == 2 ? n0 : 0)] = names;
82         }
83         if (names < (int)sample.size())
84             order = sort_suffixes(move(reduced), names);
85         else
86             for (int i = 0; i < (int)reduced.size(); i++)
87                 order[reduced[i] - 1] = i;

```

恢复采样排名并合并（接上页同一结构体）：

```

88     vector<int> rank(n + 3);
89     for (int k = 0; k < (int)order.size(); k++)
90     {
91         int i = order[k];
92         int p = i < n0 ? 3 * i + 1 : 3 * (i - n0) + 2;
93         sample[k] = p;
94         rank[p] = k + 1;
95         if (p % 3 == 1)
96             zero.push_back(p - 1);
97     }
98     radix(zero, 0);
99     vector<int> result;
100    int a = extra, b = 0;
101    while (a < (int)sample.size() && b < (int)zero.size())
102    {
103        int i = sample[a], j = zero[b];
104        bool first;
105        if (i % 3 == 1)
106            first = pair{s[i], rank[i + 1]} < pair{s[j], rank[j + 1]};
107        else
108            first = tuple{s[i], s[i + 1], rank[i + 2]}
109                    < tuple{s[j], s[j + 1], rank[j + 2]};
110        if (first)
111            result.push_back(sample[a++]);
112        else
113            result.push_back(zero[b++]);
114    }
115    while (a < (int)sample.size())
116        result.push_back(sample[a++]);
117    while (b < (int)zero.size())
118        result.push_back(zero[b++]);
119    return result;
120 }
121 };

```

模板题使用：Library Checker suffixarray (DC3)

题意：输入长度 1..500000 的小写字符串，输出所有非空后缀按字典序排列的 0-based 起点。DC3 独立程序，无需附加哨兵；原倍增或 SA-IS 用法的线上记录不转移到本程序。

所需模板：DC3 (第 31 页, 4.1.6 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://judge.yosupo.jp/problem/suffixarray>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     DC3 d(s);
8     for (int i = 0; i < (int)d.sa.size(); i++)
9         cout << d.sa[i] << (i + 1 == (int)d.sa.size() ? '\n' : ' ');
10    return 0;
11 }
```

接口用法演示：稠密整数后缀数组、逆排名及 LCP

演示约定：输入 $n, \text{alphabet}$ 与 n 个 $[0, \text{alphabet})$ 整数，分别输出 sa 、 rk 、 lcp 三行。允许 $n=0$ ，此时仍输出三个空行。 $\text{lcp}[0]=0$ ，其余对当前排名与前一名后缀比较；输入符号 0 可用，不应手工插入空后缀哨兵。API 演示，不是线上题目或 AC。

所需模板：DC3（第 31 页，4.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://oi-wiki.org/string/sa/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, alphabet;
6     cin >> n >> alphabet;
7     vector<int> s(n);
8     for (int &c : s)
9         cin >> c;
10    DC3 d(s, alphabet);
11    for (int x : d.sa)
12        cout << x << ' ';
13    cout << '\n';
14    for (int x : d.rk)
15        cout << x << ' ';
16    cout << '\n';
17    for (int x : d.lcp)
18        cout << x << ' ';
19    cout << '\n';
20    return 0;
21 }
```

第五章 后缀自动机 (SAM)

5.1 后缀自动机

extend(c) 接收整数 0..25, 每次追加一个小写字符; last 为整串状态, a[0] 为根, 根 link=-1。a[u].len 为状态最大长度, 非根代表 (len[link[u]],len[u]) 且具有同一 endpos 集合。occ 是原始种子: 真实扩展为 1、克隆为 0, 不能直接当出现次数。counts() 按长度倒序汇总后返回副本, 不改结构, 重复查询及查询后继续追加安全; 根计数为 n, 非空子串只看 u>0。distinct() 为不同非空子串数。重赋值 SuffixAutomaton() 清空; 固定 26 字母表构建 O(n), counts/distinct 为 O(n), 空间 O(26n)。如需前缀位置对应状态, 在每次 extend 后保存 last; 本结构不输出广义 SAM 或最小 DFA 状态数。

```
1 struct SuffixAutomaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
```

接上页同一结构体：

```
13 void extend(int c)
14 {
15     assert(0 <= c && c < 26);
16     int cur = (int)a.size();
17     a.push_back(Node{});
18     a[cur].len = a[last].len + 1;
19     a[cur].occ = 1;
20     int p = last;
21     while (p != -1 && !a[p].go[c])
22     {
23         a[p].go[c] = cur;
24         p = a[p].link;
25     }
26     if (p == -1)
27         a[cur].link = 0;
28     else
29     {
30         int q = a[p].go[c];
31         if (a[p].len + 1 == a[q].len)
32             a[cur].link = q;
33         else
34         {
35             int clone = (int)a.size();
36             Node copy = a[q];
37             a.push_back(copy);
38             a[clone].len = a[p].len + 1;
39             a[clone].occ = 0;
40             while (p != -1 && a[p].go[c] == q)
41             {
42                 a[p].go[c] = clone;
43                 p = a[p].link;
44             }
45             a[q].link = a[cur].link = clone;
46         }
47     }
48     last = cur;
49 }
```

接上页同一结构体：

```

51     vector<long long> counts() const
52     {
53         vector<int> bucket(a[last].len + 1), order(a.size());
54         vector<long long> ans(a.size());
55         for (auto &v : a) ++bucket[v.len];
56         partial_sum(bucket.begin(), bucket.end(), bucket.begin());
57         for (int i = 0; i < (int)a.size(); i++)
58         {
59             order[--bucket[a[i].len]] = i;
60             ans[i] = a[i].occ;
61         }
62         for (int i = (int)order.size() - 1; i > 0; i--)
63             ans[a[order[i]].link] += ans[order[i]];
64         return ans;
65     }
66
67     long long distinct() const
68     {
69         long long ans = 0;
70         for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
71         return ans;
72     }
73 };

```

模板题使用：Library Checker number_of_substrings

题意：给小写字母串，求不同非空子串的个数。extend 接收 0..25 的整数，故字符减去 a；所有字符追加后调用 distinct，结果使用 long long。

所需模板：[SuffixAutomaton](#)（第 36 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      string s;
6      cin >> s;
7      SuffixAutomaton sam;
8      for (char c : s) sam.extend(c - 'a');
9      cout << sam.distinct() << '\n';
10 }

```

模板题使用：Luogu P3804

题意：长度至 100 万的小写串，只考虑出现次数大于 1 的子串，最大化长度乘次数。counts 按长度汇总，取非根状态 $\text{len} * \text{count}$ ；无重复子串输出 0，乘积使用 long long。

所需模板：SuffixAutomaton（第 36 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3804>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     SuffixAutomaton sam;
8     for (char c : s) sam.extend(c - 'a');
9     auto count = sam.counts();
10    long long answer = 0;
11    for (int u = 1; u < (int)sam.a.size(); u++)
12    {
13        if (count[u] > 1) answer = max(answer, count[u] * sam.a[u].len);
14    }
15    cout << answer << '\n';
16 }
```

接口用法演示：区间不同子串 (kuangbin HDU4622 模型)

演示约定：来源协议：T 组，每组非空小写字母串 s 、 q 及 q 个 1-based 闭区间 $[l, r]$ ，输出该区间内不同非空子串数量。每个左端点新建 SAM，每次 extend 后累加新 last 的 len 减 link.len ；克隆只重分配旧子串，不另增贡献。固定 26 字母表，预处理 $O(n^2)$ ，查询 $O(1)$ ，答案表 $O(n^2)$ 、临时 SAM $O(n)$ 空间。不要在内层循环调用扫描所有状态的 $\text{distinct}()$ ，也不能跨左端点复用旧 SAM。当前为来源模型/API 用法，官方题面、资源限制及线上 AC 未核验。

所需模板：SuffixAutomaton (第 36 页，5.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://acm.hdu.edu.cn/showproblem.php?pid=4622>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int t;
6     cin >> t;
7     while (t--)
8     {
9         string s;
10        cin >> s;
11        int n = (int)s.size();
12        vector<vector<long long>> ans(n, vector<long long>(n));
13        for (int l = 0; l < n; l++)
14        {
15            SuffixAutomaton sam;
16            long long sum = 0;
17            for (int r = l; r < n; r++)
18            {
19                sam.extend(s[r] - 'a');
20                int u = sam.last;
21                sum += sam.a[u].len - sam.a[sam.a[u].link].len;
22                ans[l][r] = sum;
23            }
24        }
25        int q;
26        cin >> q;
27        while (q--)
28        {
29            int l, r;
30            cin >> l >> r;
31            cout << ans[l - 1][r - 1] << '\n';
32        }
33    }
34 }
```

5.2 SAM 字典序子串查询

SAMLex(sam,w={}) 引用已建好的 SuffixAutomaton 或已 build 的 GeneralSAM; 索引使用期间 sam 必须存活且保持不变。默认每个非根状态权重 1, 得到不同子串; 传单串 sam.counts() 按出现次数计数。自定义 w 长度为状态数、非负 long long, 每条到达该状态的串有 w[u] 份, 根权重强制 0。sum[u] 为本状态及后续路径权重和, 超过 LLONG_MAX 截断, 仍正确支持所有 $1 \leq k \leq \text{LLONG_MAX}$ 。kth(k,alphabet) 返回 optional<string>, 不足或 $k \leq 0$ 为 nullopt; 默认 a..z, alphabet 必须是 26 字母的排列。短串先于其扩展, 支持重复查询与自定义字母序。构建 $O(26V+L)$, 空间 $O(V+L)$, L 为最长串长; 单次查询 $O(26 \times \text{答案长度})$, 额外空间为答案。weight/sum 只读, 不支持扩展 SAM 后沿用旧索引。

```

1 // Keep sam alive and unchanged while this index is used.
2 template <class S>
3 struct SAMLex
4 {
5     const S &sam;
6     vector<long long> weight, sum;
7
8     SAMLex(const S &sam, vector<long long> w = {}) : sam(sam), weight(move(w))
9     {
10         int n = sam.a.size(), length = 0;
11         if (weight.empty()) weight.assign(n, 1);
12         assert((int)weight.size() == n);
13         weight[0] = 0;
14         sum = weight;
15         for (auto &v : sam.a) length = max(length, v.len);
16         vector<int> bucket(length + 1), order(n);
17         for (auto &v : sam.a) bucket[v.len]++;
18         for (int i = 1; i <= length; i++) bucket[i] += bucket[i - 1];
19         for (int i = 0; i < n; i++) order[--bucket[sam.a[i].len]] = i;
20         for (int i = n - 1; i >= 0; i--)
21         {
22             int u = order[i];
23             for (int v : sam.a[u].go)
24                 if (v) sum[u] += min(sum[v], LLONG_MAX - sum[u]);
25         }
26     }

```

字典序查询（接上页同一结构体）：

```

28 // alphabet must be a permutation of a..z; k is 1-based.
29 optional<string> kth(long long k, const string &alphabet = "abcdefghijklmnopqrstuvwxyz") const
30 {
31     if (k <= 0 || k > sum[0]) return nullopt;
32     int u = 0;
33     string ans;
34     while (k > weight[u])
35     {
36         k -= weight[u];
37         for (char c : alphabet)
38         {
39             int v = sam.a[u].go[c - 'a'];
40             if (!v) continue;
41             if (k > sum[v]) k -= sum[v];
42             else
43             {
44                 ans += c;
45                 u = v;
46                 break;
47             }
48         }
49     }
50     return ans;
51 }
52 };

```

应用题补充：Luogu P3975 / 弦论

题意：小写串长至 500000， $t=0$ 时相同子串只计一次， $t=1$ 按出现次数计； $1 \leq k \leq 10^9$ 。用 counts 作为状态权重，根权重由索引清零；不足输出-1。竞赛应用，不计正式模板题。

所需模板：[SuffixAutomaton](#)（第 36 页，5.1 节）、[SAMLex](#)（第 41 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3975>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     int multiple;
7     long long k;
8     cin >> s >> multiple >> k;
9     SuffixAutomaton sam;
10    for (char c : s) sam.extend(c - 'a');
11    SAMLex index(sam, multiple ? sam.counts() : vector<long long>{});
12    auto ans = index.kth(k);
13    cout << (ans ? *ans : "-1") << '\n';
14 }

```

接口用法演示：广义 SAM 查询 / API 示例

演示约定：读 n 个非空小写文档，再读 26 字母排列、1-based k 及非空查询文本 t ；输出文档子串并集的第 k 小串（不足-1），然后输出 t 与任一文档 LCS 在 t 内的起点和长度。不计文档重复次数，不求所有文档交集；示例 $n=0$ 合法。

所需模板：[GeneralSAM](#)（第 46 页，6.1 节）、[SAMLex](#)（第 41 页，5.2 节）、[sam_lcs](#)（第 44 页，5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://github.com/gsh20040816/CPCTemplate>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     GeneralSAM sam;
8     for (int i = 0; i < n; i++)
9     {
10         string s;
11         cin >> s;
12         sam.add(s);
13     }
14     sam.build();
15     string alphabet, t;
16     long long k;
17     cin >> alphabet >> k >> t;
18     SAMLex index(sam);
19     auto ans = index.kth(k, alphabet);
20     cout << (ans ? *ans : "-1") << '\n';
21     auto [start, length] = sam_lcs(sam, t);
22     cout << start << ' ' << length << '\n';
23 }
```

5.3 SAM 最长公共子串

`sam_lcs(sam,t)` 接收已有单串 SuffixAutomaton 或已 build 的 GeneralSAM 及小写文本 `t`, 返回 `t` 中的 {0-based 起点, 长度}; 多解取 `t` 中最早起点, 无公共非空串返回 {0,0}。广义 SAM 时求 `t` 与任一已加入文档的最长公共子串, 不是所有文档公共串。`t` 可空, 长度 $< \text{INT_MAX}$; 构建 SAM 的成本另计, 查询 $O(|t|)$ 、额外空间 $O(1)$, 输入与 SAM 不变。沿后缀链接回退并同步匹配长度, 不返回源文档坐标; 需实际串时用 `t.substr(start,length)`。

```

1 // Return {start, length} in t; ties choose the earliest start.
2 template <class S>
3 pair<int, int> sam_lcs(const S &sam, const string &t)
4 {
5     int p = 0, length = 0;
6     pair<int, int> ans{0, 0};
7     for (int i = 0; i < (int)t.size(); i++)
8     {
9         int c = t[i] - 'a';
10        assert(0 <= c && c < 26);
11        while (p && !sam.a[p].go[c])
12        {
13            p = sam.a[p].link;
14            length = sam.a[p].len;
15        }
16        if (sam.a[p].go[c])
17        {
18            p = sam.a[p].go[c];
19            length++;
20        }
21        else length = 0;
22        if (length > ans.second) ans = {i - length + 1, length};
23    }
24    return ans;
25 }

```

应用题补充：SPOJ LCS / 最长公共子串

题意：两行小写字母串，各长 ≤ 250000 ，输出最长公共子串长度，无公共串输出 0。getline 保留空行，并兼容 CRLF。返回位置属于第二个串，本用法只输出长度；线上 AC 和时间/内存限制待核验。

所需模板：[SuffixAutomaton](#)（第 36 页，5.1 节）、[sam_lcs](#)（第 44 页，5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/SP1811>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s, t;
6     getline(cin, s);
7     getline(cin, t);
8     if (!s.empty() && s.back() == '\r') s.pop_back();
9     if (!t.empty() && t.back() == '\r') t.pop_back();
10    SuffixAutomaton sam;
11    for (char c : s) sam.extend(c - 'a');
12    cout << sam_lcs(sam, t).second << '\n';
13 }
```

联合使用示例见第 43 页（广义 SAM 查询 / API 示例），其中直接调用本模板。

第六章 广义后缀自动机

6.1 广义后缀自动机：Trie BFS 构建

先 `add(s)` 登记若干小写字符串，再 `build()` 原地将 Trie 变为广义 SAM；`build` 后禁止继续 `add`，可重复 `build`，重新收集串需新建对象。空串与重复串允许，`distinct()` 返回所有输入串中不同非空子串的总数，不包含跨串拼接。根为 0，`a[v].go[c]` 为转移、0 表示无边，`link` 为后缀链接、`len` 为最长代表串长度；从根沿字符转移可判定子串是否存在。`a.size()` 为按文档 `endpos` 等价类构造的状态数，不应直接称为后缀并集语言的最小 DFA 点数；`ab`、`b` 是两者不同的例子。P6139 官方样例一要求 `endpos` 点数 10，后缀语言最小化仅 7 点；该题用 `a.size()`，勿再最小化。固定 26 字符，设累计输入长度 `L`、Trie 点数 `K`，登记总时间 $O(L)$ ，`build` 时间与空间 $O(26K)$ ，状态少于 $2K$ ；累计输入长度及 $2K$ 须可由 `int` 表示。克隆只复制已处理转移，不能带入尚未 BFS 处理的 Trie 子边。此版不提供出现次数或各串所属标记。新增 `add(p,c)` 在 `build` 前按 Trie 边登记， $0 \leq c < 26$ ，`p` 为已有 Trie 节点；返回子节点号，重复边复用。单次均摊 $O(1)$ ，避免把输入树展开成多条长串；`add(s)` 复用此入口。原 Trie 编号在 `build` 后仍代表原根路径，但禁止继续 `add`。

```
1 struct GeneralSAM
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7     };
8
9     vector<Node> a{Node{}};
10    bool ready = false;
11
12    // Trie step before build; repeated edges return the same node.
13    int add(int p, int c)
14    {
15        assert(!ready && 0 <= p && p < (int)a.size());
16        assert(0 <= c && c < 26);
17        if (!a[p].go[c])
18        {
19            int v = a.size();
20            a.push_back(Node{});
21            a[p].go[c] = v;
22        }
23        return a[p].go[c];
24    }
25
26    void add(const string &s)
27    {
28        assert(!ready);
29        int p = 0;
30        for (char ch : s) p = add(p, ch - 'a');
31    }
```

BFS 构建（接上页同一结构体）：

```

33 void build()
34 {
35     if (ready) return;
36     a.reserve(2 * a.size());
37     queue<pair<int, int>> todo;
38     for (int c = 0; c < 26; c++)
39         if (a[0].go[c]) todo.push({0, c});
40     while (!todo.empty())
41     {
42         auto [parent, c] = todo.front();
43         todo.pop();
44         int cur = a[parent].go[c], p = a[parent].link;
45         a[cur].len = a[parent].len + 1;
46         while (p != -1 && !a[p].go[c])
47         {
48             a[p].go[c] = cur;
49             p = a[p].link;
50         }
51         if (p == -1)
52             a[cur].link = 0;
53         else
54         {
55             int q = a[p].go[c];
56             if (a[p].len + 1 == a[q].len)
57                 a[cur].link = q;
58             else
59             {
60                 int clone = a.size();
61                 Node copy = a[q];
62                 copy.len = a[p].len + 1;
63                 for (int &v : copy.go)
64                     if (!a[v].len) v = 0;
65                 a.push_back(copy);
66                 while (p != -1 && a[p].go[c] == q)
67                 {
68                     a[p].go[c] = clone;
69                     p = a[p].link;
70                 }
71                 a[q].link = a[cur].link = clone;
72             }
73         }
74         for (int ch = 0; ch < 26; ch++)
75             if (int v = a[cur].go[ch]; v && !a[v].len) todo.push({cur, ch});
76     }
77     ready = true;
78 }

```

不同子串计数（接上页同一结构体）：

```
80     long long distinct() const
81     {
82         assert(ready);
83         long long ans = 0;
84         for (int v = 1; v < (int)a.size(); v++) ans += a[v].len - a[a[v].link].len;
85         return ans;
86     }
87 };
```

模板题使用：Library Checker number_of_substrings

题意：本题只有一个字符串，输出不同非空子串数。广义模板可在 build 前多次 add；本例验证其单串特例，多串行为另有独立对拍。若只处理单串，原 SuffixAutomaton 更合适。此题不验证广义 SAM 状态数或最小 DFA 状态数。

所需模板：GeneralSAM（第 46 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     GeneralSAM sam;
8     sam.add(s);
9     sam.build();
10    cout << sam.distinct() << '\n';
11 }
```

模板题使用：Luogu P6139

题意：读入所有小写字符串，求不同非空子串数及含根的广义 SAM 点数。先全部 add，再 build，输出 distinct 和 a.size；总长度至 100 万、文档数至 40 万。点数按官方样例的 endpos 等价类口径：aa、ab、bac、caa 输出 10 和 10，不另做后缀语言 DFA 最小化。本例通过完整驱动本地验证，线上待补。

所需模板：GeneralSAM（第 46 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P6139>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     GeneralSAM sam;
8     while (n--)
9     {
10         string s;
11         cin >> s;
12         sam.add(s);
13     }
14     sam.build();
15     cout << sam.distinct() << '\n' << sam.a.size() << '\n';
16 }
```

接口用法演示：直接 Trie 建图与含空串排名 / API 示例

演示约定：输入 n, q ，再给 n 条 parent, c ($0 \leq \text{parent} < i$)，节点 0 为空根；重复同标签边可合并。一次 BFS 构建后输出含空串的不同子串数。每次输入字母序排列和 1-based k ， $k=1$ 输出空行，其余调用 $\text{kth}(k-1)$ ，不足输出 -1。自行约定的 API 演示，不冒充 TSUBSTR 官方驱动。

所需模板：GeneralSAM (第 46 页, 6.1 节)、SAMLex (第 41 页, 5.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://github.com/gsh20040816/CPCTemplate>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     GeneralSAM sam;
8     vector<int> state(n + 1);
9     for (int i = 1; i <= n; i++)
10    {
11        int parent;
12        char c;
13        cin >> parent >> c;
14        state[i] = sam.add(state[parent], c - 'a');
15    }
16    sam.build();
17    cout << sam.distinct() + 1 << '\n';
18    SAMLex index(sam);
19    while (q--)
20    {
21        string alphabet;
22        long long k;
23        cin >> alphabet >> k;
24        if (k == 1) cout << '\n';
25        else
26        {
27            auto ans = index.kth(k - 1, alphabet);
28            cout << (ans ? *ans : "-1") << '\n';
29        }
30    }
31 }

```

应用题补充：CodeChef TSUBSTR / Substrings on a Tree

题意：给以 1 为根的无向树及每个顶点的小写字母，求所有祖先到后代路径串中不同串的数量，包含空串；每次按给定 26 字母排列求第 k 小，不足输出 -1， $k=1$ 输出空行。N 至 250000、Q 至 50000， k 至 $2^{63}-1$ ，总输出不超过 800KB，原题字母随机。根顶点字母也必须登记；重复标签路由 Trie 合并。先按树做 BFS 登记，再构建 SAM，不展开根叶字符串。

所需模板：GeneralSAM（第 46 页，6.1 节）、SAMLex（第 41 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.codechef.com/problems/TSUBSTR>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     string s;
7     cin >> n >> q >> s;
8     vector<vector<int>> g(n);
9     for (int i = 1; i < n; i++)
10     {
11         int u, v;
12         cin >> u >> v;
13         u--;
14         v--;
15         g[u].push_back(v);
16         g[v].push_back(u);
17     }
18     GeneralSAM sam;
19     vector<int> par(n, -1), state(n);
20     state[0] = sam.add(0, s[0] - 'a');
21     queue<int> todo;
22     todo.push(0);
23     while (!todo.empty())
24     {
25         int u = todo.front();
26         todo.pop();
27         for (int v : g[u])
28         {
29             if (v == par[u]) continue;
30             par[v] = u;
31             state[v] = sam.add(state[u], s[v] - 'a');
32             todo.push(v);
33         }
34     }

```

继续处理操作（接上页同一 main）：

```
35     sam.build();
36     cout << sam.distinct() + 1 << '\n';
37     SAMLex index(sam);
38     while (q--)
39     {
40         string alphabet;
41         long long k;
42         cin >> alphabet >> k;
43         if (k == 1) cout << '\n';
44         else
45         {
46             auto ans = index.kth(k - 1, alphabet);
47             cout << (ans ? *ans : "-1") << '\n';
48         }
49     }
50 }
```

相关应用：指定状态在线扩展；任意插入顺序不保证 Trie 节点数线性复杂度。

6.2 指定状态在线扩展 SAM

OnlineSAM() 以 0 为根；extend(p,c) 给状态 p 代表的最长串末尾接字符 $0 \leq c < 26$ ，返回该串的状态号，调用后立即可沿 go 判断已有子串。旧状态的 len 与编号保持，但 link/go 可变化；不可用短于 len[p] 的匹配串冒充该最长串。逐串输入时每串从 p=0 开始；重复路径允许。total 为当前不同非空子串数，a 含根，公开字段只读；不自动统计出现次数、文档频率或终态。一次至多增加 2 状态，需状态数和长度可用 int 索引，数组可分配。单次 $O(\text{当前最大串长 } D + 26)$ ，Q 次保守 $O(QD + 26Q)$ ，任意顺序可能平方级；已知整棵 Trie 优先 GeneralSAM 的 BFS 构造。空间 $O(26Q)$ ，重置赋新对象。内部递归只进入已有转移分支，深度至多 2；构建完成后可用 SAMLex/sam_lcs，继续扩展前先弃用旧查询索引。

```
1 struct OnlineSAM
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7     };
8
9     vector<Node> a{Node{}};
10    long long total = 0;
```

指定状态在线扩展（接上页同一结构体）：

```

12 // Append c to the longest string represented by state p.
13 int extend(int p, int c)
14 {
15     assert(0 <= p && p < (int)a.size() && 0 <= c && c < 26);
16     if (int q = a[p].go[c])
17     {
18         if (a[q].len == a[p].len + 1) return q;
19         int clone = a.size();
20         Node copy = a[q];
21         copy.len = a[p].len + 1;
22         a.push_back(copy);
23         a[q].link = clone;
24         while (p != -1 && a[p].go[c] == q)
25         {
26             a[p].go[c] = clone;
27             p = a[p].link;
28         }
29         return clone;
30     }
31     int cur = a.size();
32     Node node;
33     node.len = a[p].len + 1;
34     a.push_back(node);
35     while (p != -1 && !a[p].go[c])
36     {
37         a[p].go[c] = cur;
38         p = a[p].link;
39     }
40     int parent = p == -1 ? 0 : extend(p, c);
41     a[cur].link = parent;
42     total += a[cur].len - a[parent].len;
43     return cur;
44 }
45 };

```

接口用法演示：在线分支扩展 / API 示例

演示约定： q 次操作，每次给历史版本 parent ($0 \leq \text{parent} < i$) 和小写字母 c ；将该版本的完整串接 c 作为新版本，版本 0 为空。保存返回状态号，每次输出当前全部版本的不同非空子串并集大小。这是添加路径，不是持久化 SAM：所有查询读取累计语言。

所需模板：OnlineSAM (第 53 页, 6.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://github.com/gsh20040816/CPCTemplate>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int q;
6     cin >> q;
7     OnlineSAM sam;
8     vector<int> state(q + 1);
9     for (int i = 1; i <= q; i++)
10    {
11        int parent;
12        char c;
13        cin >> parent >> c;
14        state[i] = sam.extend(state[parent], c - 'a');
15        cout << sam.total << '\n';
16    }
17 }
```

模板题使用：Luogu P6139 / 在线扩展版本

题意：每个文档从根重新扩展，输出不同非空子串数和含根 `endpos` 状态数；题面最多 40 万串、总长至 100 万。与已有 GeneralSAM 相同的状态口径，不做后缀语言 DFA 最小化；本地验证，线上待补。任意父状态 API 的最坏复杂度不能据此称线性。

所需模板：OnlineSAM (第 53 页, 6.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P6139>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     OnlineSAM sam;
8     for (int i = 0; i < n; i++)
9     {
10        string s;
11        cin >> s;
12        int p = 0;
13        for (char c : s) p = sam.extend(p, c - 'a');
14    }
15    cout << sam.total << '\n';
16    cout << sam.a.size() << '\n';
17 }
```

6.3 SAM 文档频率与阈值后缀

代码依赖：[Fenwick](#)（第 78 页，C.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

`SAMDocuments(sam,s)` 接收已完成构建的 `GeneralSAM` 及同一批小写文档 `s`；也兼容按该文档集完成的 `OnlineSAM` 或单文档 `SuffixAutomaton`。允许空串和重复串，重复输入按不同文档计数。索引期间 `sam` 须存活且不变。`cnt[u]` 是该状态任一非空代表串出现的文档数，`cnt[0]=s.size()` 约定空串出现在每篇文档；不能把单串出现次数当文档数。`lengths(k)` 要求 $k \geq 1$ ，返回每个状态最长代表串中、出现于至少 k 篇文档的最长后缀长度，无非空解为 0； k 大于文档数全 0。查询任意非空模式串时，从根转移成功后读取 `cnt`，缺边返回 0。 L 为文档总长， V 为状态数， D 为文档数：构造 $O((L+V)\log(L+2)+D)$ ，空间 $O(L+V+D)$ ，每次 `lengths` 为 $O(V)$ 。递归后缀链接树深度不超过最长文档长度加 1；保留递归，不模拟开栈。 L 、 D 、 V 须小于 `INT_MAX`，数组可分配；`a/g/cnt` 视为只读。

```
1 template <class S>
2 struct SAMDocuments
3 {
4     const S &sam;
5     vector<vector<int>>> g;
6     vector<int> cnt;
```

文档标记与子树去重（接上页同一结构体）：

```

8      SAMDocuments(const S &sam, const vector<string> &s)
9          : sam(sam), g(sam.a.size()), cnt(sam.a.size())
10     {
11         int n = sam.a.size(), size = 0;
12         vector<vector<int>> tag(n);
13         for (int u = 1; u < n; u++) g[sam.a[u].link].push_back(u);
14         for (int id = 0; id < (int)s.size(); id++)
15         {
16             int p = 0, len = 0;
17             for (char c : s[id])
18             {
19                 assert('a' <= c && c <= 'z');
20                 p = sam.a[p].go[c - 'a'];
21                 len++;
22                 assert(p && sam.a[p].len == len);
23                 tag[p].push_back(id);
24                 size++;
25             }
26         }
27         Fenwick<int> bit(size);
28         vector<int> last(s.size());
29         int timer = 0;
30         auto dfs = [&](auto &&self, int u) -> void
31         {
32             int l = timer + 1;
33             for (int id : tag[u])
34             {
35                 if (last[id]) bit.add(last[id], -1);
36                 last[id] = ++timer;
37                 bit.add(timer, 1);
38             }
39             for (int v : g[u]) self(self, v);
40             cnt[u] = bit.query(l, timer);
41         };
42         dfs(dfs, 0);
43         cnt[0] = s.size();
44     }

```

阈值后缀长度（接上页同一结构体）：

```
46 // Longest suffix length occurring in at least k documents, for each state.
47 vector<int> lengths(int k) const
48 {
49     assert(k >= 1);
50     vector<int> best(cnt.size());
51     auto dfs = [&](auto &&self, int u) -> void
52     {
53         if (cnt[u] >= k) best[u] = sam.a[u].len;
54         for (int v : g[u])
55         {
56             best[v] = best[u];
57             self(self, v);
58         }
59     };
60     dfs(dfs, 0);
61     return best;
62 }
63 };
```

应用题补充：Codeforces 204E / Little Elephant and Strings

题意：给 n 篇非空小写文档和 k ，分别求每篇中出现于至少 k 篇文档的子串区间个数；重复文档分别计数，区间重复出现也分别计数。 n 、 k 及总长度至 100000， k 允许大于 n 。先统计文档频率，再按每个前缀累加合格最长后缀长度；答案用 long long。比赛应用，不登记为非比赛模板题。

所需模板：GeneralSAM (第 46 页, 6.1 节)、Fenwick (第 78 页, C.1 节)、SAMDocuments (第 56 页, 6.3 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://codeforces.com/problemset/problem/204/E>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, k;
6     cin >> n >> k;
7     vector<string> s(n);
8     GeneralSAM sam;
9     for (auto &t : s)
10    {
11        cin >> t;
12        sam.add(t);
13    }
14    sam.build();
15    SAMDocuments index(sam, s);
16    auto best = index.lengths(k);
17    for (int id = 0; id < n; id++)
18    {
19        int p = 0;
20        long long ans = 0;
21        for (char c : s[id])
22        {
23            p = sam.a[p].go[c - 'a'];
24            ans += best[p];
25        }
26        cout << ans << (id + 1 == n ? '\n' : ' ');
27    }
28 }
```

接口用法演示：子串文档频率 / API 示例

演示约定：输入 n, q 、 n 篇非空小写文档和 q 个非空小写模式串，输出每个模式出现于多少篇输入文档；同一文档内重复出现只算一次，相同输入文档分别计数。走转移失败立即返回 0。本例 n 可为 0；核心另支持空文档，空串频率为 `cnt[0]`。

所需模板：[GeneralSAM](#) (第 46 页, 6.1 节)、[Fenwick](#) (第 78 页, C.1 节)、[SAMDocuments](#) (第 56 页, 6.3 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

来源：<https://github.com/gsh20040816/CPCTemplate>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     vector<string> s(n);
8     GeneralSAM sam;
9     for (auto &t : s)
10    {
11        cin >> t;
12        sam.add(t);
13    }
14    sam.build();
15    SAMDocuments index(sam, s);
16    while (q--)
17    {
18        string t;
19        cin >> t;
20        int p = 0;
21        for (char c : t)
22        {
23            p = sam.a[p].go[c - 'a'];
24            if (!p) break;
25        }
26        cout << (p ? index.cnt[p] : 0) << '\n';
27    }
28 }
```

第七章 Manacher

7.1 Manacher 回文半径

manacher(s) 返回 odd/even, 下标从 0 开始。odd[i] 包含中心, 闭区间 [i-odd[i]+1,i+odd[i]-1], 长度 2*odd[i]-1; even[i] 的中心在 i 前, 区间 [i-even[i],i+even[i]-1], 长度 2*even[i], 零半径为空。空串合法, 支持任意字节 (含零), 按字节而非 Unicode 字素比较; int 索引要求 $n < 2^{30}$ 。最长回文取上述长度最大值; 全部 odd/even 以 long long 求和得到回文出现总数。O(n) 时间和空间, 边界显式检查, 无需选择哨兵字符。输入为 string、vector<int> 或 vector<long long> 对象, 支持空序列和任意整数; 泛型 S 需 size()、随机下标及 O(1) 相等比较。字符串字面量请显式写 string("abc")。输入不拷贝, 输出移动返回。

```
1 // odd[i] includes the center; even[i] is centered before i.
2 template <class S>
3 pair<vector<int>, vector<int>> manacher(const S &s)
4 {
5     int n = (int)s.size();
6     vector<int> odd(n), even(n);
7     for (int i = 0, l = 0, r = -1; i < n; i++)
8     {
9         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
10        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
11        odd[i] = k--;
12        if (i + k > r)
13        {
14            l = i - k;
15            r = i + k;
16        }
17    }
18    for (int i = 0, l = 0, r = -1; i < n; i++)
19    {
20        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
21        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
22        even[i] = k--;
23        if (i + k > r)
24        {
25            l = i - k - 1;
26            r = i + k;
27        }
28    }
29    return {move(odd), move(even)};
30 }
```

模板题使用：Library Checker enumerate_palindromes

题意：给非空字符串，从左到右输出每个字符及相邻字符间隙处的最长回文长度，共 $2n-1$ 项。odd 含中心，even[i] 的中心在 i 前面，因此分别转成 $2*odd[i]-1$ 和 $2*even[i]$ 。

所需模板：manacher（第 61 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_palindromes

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     int n = s.size();
8     auto [odd, even] = manacher(s);
9     vector<int> answer(2 * n - 1);
10    for (int i = 0; i < n; i++)
11    {
12        answer[2 * i] = 2 * odd[i] - 1;
13        if (i) answer[2 * i - 1] = 2 * even[i];
14    }
15    for (int i = 0; i < (int)answer.size(); i++)
16        cout << answer[i] << (i + 1 == (int)answer.size() ? '\n' : ' ');
17 }
```

模板题使用：Luogu P3805

题意：给定长度不超过 11000000 的小写字符串，输出最长回文子串长度。odd 含中心，even[i] 以 i 前间隙为中心，分别用 $2*odd[i]-1$ 和 $2*even[i]$ 取最大值。

所需模板：manacher（第 61 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3805>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     auto [odd, even] = manacher(s);
8     int answer = 0;
9     for (int i = 0; i < (int)s.size(); i++)
10    {
11        answer = max(answer, 2 * odd[i] - 1);
12        answer = max(answer, 2 * even[i]);
13    }
14    cout << answer << '\n';
15 }
```

接口用法演示：整数序列回文半径 / API 示例

演示约定：输入 n 及 n 个 signed64 整数，输出两行 odd/even 半径，均 0-based；odd 含中心，even 中心在 i 之前。 $n=0$ 输出两空行。接口示例，不对应正式模板题或线上 AC。

所需模板：[manacher](#) (第 61 页, 7.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://github.com/gsh20040816/CPCtemplate>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     vector<long long> a(n);
8     for (auto &x : a) cin >> x;
9     auto [odd, even] = manacher(a);
10    for (int x : odd) cout << x << ' ';
11    cout << '\n';
12    for (int x : even) cout << x << ' ';
13    cout << '\n';
14 }
```

第八章 回文树

8.1 回文树 PAM: 回文后缀与出现次数

`add(ch)` 追加小写英文字母，返回最长回文后缀节点，亦保存于 `last`。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；`distinct()` 为不同非空回文串数。`t[u].len` 为长度，`fail` 为最长真回文后缀链接，`suffixes` 为该节点的非空回文后缀数量；追加后的 `t[last].suffixes` 就是本位置结尾的回文串数。`total` 为当前全部回文子串的出现次数之和。`end` 为首次出现时的 0-based 右端点，可恢复原串子段；`hits` 仅统计作为最长后缀的次数，不能当出现次数。`occurrences()` 返回按节点编号的完整出现次数副本，根置零， $O(n)$ ；重复调用以及查询后继续追加均安全。重新赋值 `PalindromicTree()` 清空重建。固定 26 字符集下建树 $O(n)$ ，空间 $O(26n)$ 。

```
1 struct PalindromicTree
2 {
3     struct Node
4     {
5         int next[26]{};
6         int len = 0, fail = 0, suffixes = 0, end = -1;
7         long long hits = 0;
8     };
9
10    vector<Node> t;
11    vector<int> s;
12    int last = 0;
13    long long total = 0;
14
15    PalindromicTree() : t(2), s(1, -1)
16    {
17        t[0].fail = 1;
18        t[1].len = -1;
19    }
20
21    int get_fail(int u) const
22    {
23        int pos = s.size() - 1;
24        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
25        return u;
26    }
```

接上页同一结构体：

```

28 // Append a lowercase letter; return the longest palindromic suffix node.
29 int add(char ch)
30 {
31     int c = ch - 'a';
32     assert(0 <= c && c < 26);
33     s.push_back(c);
34     int u = get_fail(last);
35     if (!t[u].next[c])
36     {
37         int v = t.size();
38         t.push_back(Node{});
39         t[v].len = t[u].len + 2;
40         t[v].fail = t[get_fail(t[u].fail)].next[c];
41         t[v].suffixes = t[t[v].fail].suffixes + 1;
42         t[v].end = (int)s.size() - 2;
43         t[u].next[c] = v;
44     }
45     last = t[u].next[c];
46     t[last].hits++;
47     total += t[last].suffixes;
48     return last;
49 }
50
51 int distinct() const { return (int)t.size() - 2; }
52
53 // Copy-and-propagate: repeated calls and later appends remain valid.
54 vector<long long> occurrences() const
55 {
56     vector<long long> count(t.size());
57     for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
58     for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
59     count[0] = 0;
60     count[1] = 0;
61     return count;
62 }
63 };

```

模板题使用：Luogu P5496

题意：按上一次答案把当前字符循环偏移后追加，输出以当前位置结尾的回文串个数。add 返回最长回文后缀节点，读取其 suffixes。

所需模板：PalindromicTree (第 64 页, 8.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5496>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     std::string s;
6     std::cin >> s;
7     PalindromicTree pam;
8     int answer = 0;
9     for (int i = 0; i < (int)s.size(); i++)
10    {
11        char ch = 'a' + (s[i] - 'a' + answer) % 26;
12        int u = pam.add(ch);
13        answer = pam.t[u].suffixes;
14        std::cout << answer << (i + 1 == (int)s.size() ? '\n' : ' ');
15    }
16 }
```

应用题补充：Luogu P3649

题意：求所有回文子串中“长度乘出现次数”的最大值。整串追加完后调用 occurrences 汇总次数，不能直接把 hits 当完整出现次数。

所需模板：PalindromicTree (第 64 页, 8.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3649>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     std::string s;
6     std::cin >> s;
7     PalindromicTree pam;
8     for (char ch : s) pam.add(ch);
9     auto count = pam.occurrences();
10    long long answer = 0;
11    for (int u = 2; u < (int)count.size(); u++)
12        answer = std::max(answer, count[u] * pam.t[u].len);
13    std::cout << answer << '\n';
14 }
```

第九章 最小表示法

9.1 最小循环表示

`minimum_rotation(s)` 返回字典序最小循环表示的起点 (0-based), 空串返回 0; 按无符号字节比较, 与 `string` 字典序一致。O(n)。

```
1 int minimum_rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if (!n) return 0;
5     while (i < n && j < n && k < n)
6     {
7         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
8         if (a == b)
9         {
10             ++k;
11             continue;
12         }
13         if (a > b)
14         {
15             i += k + 1;
16             if (i == j) ++i;
17         }
18         else
19         {
20             j += k + 1;
21             if (i == j) ++j;
22         }
23         k = 0;
24     }
25     return min(i, j);
26 }
```

模板题使用：Luogu P13270 【模板】最小表示法

题意：给定长度 $1..10^7$ 的小写字母字符串，输出所有循环位移中字典序最小的字符串。minimum_rotation 返回 0-based 起点，使用 rotate 原地重排后输出； $O(n)$ 时间，算法本体 $O(1)$ 额外空间。

所需模板：minimum_rotation (第 67 页, 9.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P13270>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     string s;
7     if (!(cin >> n >> s)) return 0;
8     int p = minimum_rotation(s);
9     rotate(s.begin(), s.begin() + p, s.end());
10    cout << s << '\n';
11 }
```

第十章 配套知识与验证范围

10.1 回文半径与在线回文计数

Manacher 的 $odd[i]$ 包含中心, 最长奇回文为 $[i - odd[i] + 1, i + odd[i] - 1]$, 长度 $2odd[i] - 1$; $even[i]$ 以 i 前间隙为中心, 最长偶回文为 $[i - even[i], i + even[i] - 1]$, 长度 $2even[i]$ 。所有 $odd/even$ 之和是回文的出现总次数, 应以 long long 累加; 这不是不同回文串数量。恢复最长子串时记录对应起点和长度即可。

附件插入分隔符后的半径有包含中心与不含中心两种约定, 不能直接套到本库数组。哨兵版本还依赖字母表: 小写字母的写法不能原样推广到含标记或零字节的输入。本库显式检查边界, 空串和任意字节均合法; 按字节而非 Unicode 字素比较, int 索引要求 $n < 2^{30}$ 。P3805 的 1100 万字符相同、交替和无长度至少二回文的三种闭式输入已通过双模式本地检查。

PAM 的 suffixes 是当前回文节点的非空回文后缀数量, 等于 fail 节点的 suffixes 加一。P5496 必须先按上一个答案解码当前字符, 再 add, 输出 $t[last].suffixes.hits$ 仅统计作为最长后缀的命中次数; 完整次数应取 occurrences() 的返回值。该函数复制后再沿 fail 汇总, 不改 hits, 因此重复查询及查询后继续追加均安全。total 为逐位置 suffixes 的总和, 最多 $n(n+1)/2$; distinct 才是不同非空回文串数。PAM 固定小写字母, 内部前置 -1 与 $0 \dots 25$ 编码不同, 不继承 Manacher 的任意字节契约。

独立逐子串逆序比较、PAM 全短串及两题最大规模驱动均在普通和 ASan/UBSan 下通过。Manacher 的 LC 全部 24 组官方数据双模式本地 checker 接受, 不等于线上 AC。详细接口与记录见仓库 docs/PALINDROME-ATTACHMENT.md。

10.2 整数后缀数组的边界与 SAM 次数

SuffixArray 的整数入口要求 $0 \leq a[i] < alphabet$; 稀疏大整数先保序离散化。sa/rk/lcp 都从零编号, lcp[k] 比较 sa[k-1] 与 sa[k], 首项为零。附件第二份倍增模板额外要求 LCP 遇分隔符停止, 不能把普通 LCP 不加处理地代入。令 remain[i] 为从 i 到下一个分隔符之前的普通字符数, 则分段前缀长为

$$height[k] = \min\{lcp[k], remain[sa[k-1]], remain[sa[k]]\}.$$

分隔符位置 remain 为零, 连续或重复分隔符同样适用; 保留原 lcp 可避免两种语义混用。两个字符串拼接后, 相同前缀的后缀在 SA 中连续, 所以找最长公共子串只需检查异来源的相邻后缀。输出时转回各自原串的半开区间, 不把分隔符当原字符。

SAM 状态 u 代表长度 $(len[link[u]], len[u])$ 内具有同一 endpos 集合的子串。真实扩展状态的 occ 种子为一, 克隆为零; 附件存放插入 id 的 endpos 字段不等于完整出现次数。counts 按长度倒序传播, 不能按创建编号倒序, 因为克隆可能后创建。查询返回副本, 不改种子, 所以重复查询后继续 extend 安全。根计数为串长 n , 不能解释成空串的 $n+1$ 个出现位置; 非空子串只看非根状态。P3804 仅在 $count[u] > 1$ 时取 $count[u] * len[u]$, 乘积用 long long, 无重复子串则为零。若需要每个前缀对应的状态, 调用者在每次 extend 后保存 last 即可。

独立后缀排序、逐子串 endpos 集合、分隔符截断与完整百万规模驱动双模式通过。LC 后缀数组 50 组、最长公共子串 28 组官方数据双模式本地 checker 接受, 不等于线上 AC 或速度排名。细节见仓库 docs/SUFFIX-ATTACHMENT.md。

10.3 广义 SAM 点数与后缀语言

GeneralSAM 的状态按文档编号与结束位置组成的 endpos 集合等价。先 add 全部小写串, 再 build; 根为零, distinct 返回不同非空子串总数, a.size() 包含根。构造后不再 add, 重复 build 安全; 重建另开对象。重复串和空串在库内合法, P6139 则要求输入非空串。

多串 `endpos` 类不能直接称为后缀并集的最小 DFA。例如 `ab`、`b` 的三个非空子串 `a`、`b`、`ab` 具有不同 `endpos` 集合，含根共四点；仅识别后缀并集时，读完 `b` 和 `ab` 的状态可合并，含初态只需三点（省略拒绝汇点）。P6139 官方样例 `aa`、`ab`、`bac`、`caa` 要求不同子串数与点数均为十，符合 `endpos` 口径；把后缀 Trie 按右语言最小化则为七点，不能据题面中的“最小”二字输出七。完整用法直接输出 `a.size()`，不额外做 DFA 合并。

独立枚举文档位置集合与两种右语言计算核对上述区别。完整驱动的 600 组随机参考及排列、百万字符链和克隆密集串、两条各五十万字符的交替串、四十万文档合计百万字符，以及深十五完整二叉 Trie 均通过普通和 ASan/UBSan。这些是本地证据，尚未新增 P6139 线上 AC 或排名。构造与口径详见仓库 `docs/GENERAL-SAM-NOTES.md`。

第一部分

跨类依赖

附录 A 数论

A.1 模算术简介

A.1.1 静态模整数

固定 int 模数 $1 \leq \text{mod} \leq \text{INT_MAX}$, 可为合数。构造接受 signed64 并正规化, v 保持 $0.. \text{mod}-1$; 不要手动破坏正规化。加减乘 $O(1)$, pow 非负 signed64 指数、时间 $O(\log(e+1))$ 。try_inv 用欧几里得求逆: $\text{gcd}(v, \text{mod})=1$ 时返回 optional 值, 否则 nullopt; $\text{mod}=1$ 返回代表 0。inv、/、/= 要求分母可逆; 断言开启时诊断违反前提, 关闭断言不承诺拒绝非法除法。逆元 $O(\log(\text{mod}+1))$ 、额外空间 $O(1)$, 只用 long long, 不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用, 支持自赋值与链式运算, 可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
1 template <int mod> struct ModInt
2 {
3     static_assert(mod >= 1);
4     int v;
5
6     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
7
8     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
9
10    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
11
12    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
13
14    ModInt pow(long long e) const
15    {
16        assert(e >= 0);
17        ModInt a = *this, r = 1;
18        for (; e >= 1, a = a * a)
19            if (e & 1) r = r * a;
20        return r;
21    }
```

求逆与除法（接上页同一结构体）：

```
23 optional<ModInt> try_inv() const
24 {
25     long long a = v, b = mod, x = 1, y = 0;
26     while (b)
27     {
28         long long q = a / b;
29         a -= q * b;
30         swap(a, b);
31         x -= q * y;
32         swap(x, y);
33     }
34     if (a != 1) return nullopt;
35     return ModInt(x);
36 }
37
38 ModInt inv() const
39 {
40     auto r = try_inv();
41     assert(r);
42     return *r;
43 } // requires an invertible residue
44
45 ModInt operator/(ModInt b) const { return *this * b.inv(); }
46
47 ModInt &operator+=(ModInt b) { return *this = *this + b; }
48
49 ModInt &operator-=(ModInt b) { return *this = *this - b; }
50
51 ModInt &operator*=(ModInt b) { return *this = *this * b; }
52
53 ModInt &operator/=(ModInt b) { return *this = *this / b; }
54 };
```

联合使用示例见第 77 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p, choose(n,k) 返回 $C(n,k)$ 模 p; n、k 为完整 unsigned long long 范围, $k > n$ 返回 0, $C(0,0)=1$ 。重新赋值 Lucas(p) 可更换模数并重新预处理; 不检测素性, 要求调用者保证 p 为素数且 $p \geq 2$ 。预处理时间 $O(p + \log p)$, 动态版空间 $O(p)$; 单次查询 $O(1 + \log_p(n+1))$ 。适合小素数模数, 通常 $p \leq 10^6$; 大素数不能因查询下标小就忽略这里整段 $O(p)$ 的预处理。fac、ifac 下标为 $0..p-1$, 模数和表只读; 中间乘法用 long long。合数模数请使用 exLucas。

```

1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p)
7     {
8         assert(p >= 2);
9         mod = p;
10        fac.resize(p);
11        ifac.resize(p);
12        fac[0] = 1;
13        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
14        long long a = fac[p - 1], r = 1;
15        for (int e = p - 2; e; e >>= 1)
16        {
17            if (e & 1) r = r * a % p;
18            a = a * a % p;
19        }
20        ifac[p - 1] = r;
21        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
22    }

```

```

24 int choose(unsigned long long n, unsigned long long k) const
25 {
26     if (k > n) return 0;
27     long long answer = 1;
28     while (n || k)
29     {
30         int a = n % mod, b = k % mod;
31         if (b > a) return 0;
32         answer = answer * fac[a] % mod;
33         answer = answer * ifac[b] % mod;
34         answer = answer * ifac[a - b] % mod;
35         n /= mod;
36         k /= mod;
37     }
38     return answer;
39 }
40 };

```

模板题使用：Luogu P3807

题意：多组查询 $C(n+m, n)$ 模 p ，题目保证 p 为素数且不超过 10^5 。每组构造 $\text{Lucas}(p)$ ，choose 的参数为 $n+m$ 、 n ；此题范围内 $n+m$ 不会溢出。

所需模板：Lucas（第 74 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3807>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int t;
6     cin >> t;
7     while (t--)
8     {
9         unsigned long long n, m;
10        int p;
11        cin >> n >> m >> p;
12        Lucas solver(p);
13        cout << solver.choose(n + m, n) << '\n';
14    }
15    return 0;
16 }
```

附录 B 组合数学

B.1 排列组合

B.1.1 阶乘组合数

代码依赖：ModInt (第 72 页, A.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 $0..n$, 要求 prime 为素数且 $0 \leq n < \text{prime}$ 。init(n) 只扩大当前上限, 较小或相同的 n 不改变表; choose(n,k) 返回 $C(n,k)$, permute(n,k) 返回 $n!/(n-k)!$ 。查询前显式预处理到 n , 查询本身不扩表。 $n < 0$, $k < 0$ 或 $k > n$ 返回 0, $C(0,0)=P(0,0)=1$ 。fac、ifac 为可直接读取的阶乘与逆阶乘表, 增长可能使旧引用失效。单次查询 $O(1)$; 从 old 扩到 n 新增 $O(n-\text{old}+\log \text{prime})$ 次域运算, 重分配还可能复制旧表。首次预处理 $O(n+\log \text{prime})$, 空间 $O(n)$ 。超过模数不能使用阶乘除法: 小素数用 Lucas, 小合数用 exLucas。

```
1 template <int mod> struct Binomial
2 {
3     using Z = ModInt<mod>;
4     vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6     Binomial(int n = 0) { init(n); }
7
8     void init(int n)
9     {
10         assert(0 <= n && n < mod);
11         int old = (int)fac.size() - 1;
12         if (n <= old) return;
13         fac.resize(n + 1);
14         ifac.resize(n + 1);
15         for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
16         ifac[n] = fac[n].inv();
17         for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
18     }
```

```
20 Z choose(int n, int k) const
21 {
22     if (k < 0 || k > n) return 0;
23     assert(n < (int)fac.size());
24     return fac[n] * ifac[k] * ifac[n - k];
25 }
26
27 Z permute(int n, int k) const
28 {
29     if (k < 0 || k > n) return 0;
30     assert(n < (int)fac.size());
31     return fac[n] * ifac[n - k];
32 }
33 };
```

应用题补充：Luogu P1313 计算系数

题意：求 $(by+ax)$ 的 k 次方中指定项的系数，模 10007。 x 、 y 的指数分别为 n 、 m ，均非负且 $n+m=k$ 。 k 至多 1000， a 、 b 均为不超过 1000000 的非负数。组合 $C(k,n)$ 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 72 页，A.1.1 节）、Binomial（第 76 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P1313>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int a, b, k, n, m;
6     cin >> a >> b >> k >> n >> m;
7     Binomial<10007> c(k);
8     auto x = ModInt<10007>(a).pow(n);
9     auto y = ModInt<10007>(b).pow(m);
10    cout << (c.choose(k, n) * x * y).v << '\n';
11 }
```

附录 C 树状数组

C.1 树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based, k th 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。 $n \leq 100000$, $|x| \leq 10000000$ ；总复杂度 $O(n \log n)$ ，空间 $O(n)$ 。

所需模板：Fenwick（第 78 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     if (!(cin >> n)) return 0;
7     vector<pair<int, int>> q(n);
8     vector<int> a;
9     for (auto &[op, x] : q)
10    {
11        cin >> op >> x;
12        if (op != 4) a.push_back(x);
13    }
14    sort(a.begin(), a.end());
15    a.erase(unique(a.begin(), a.end()), a.end());
16    Fenwick<int> f(a.size());
17    for (auto [op, x] : q)
18    {
19        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
20        if (op == 1)
21            f.add(p + 1, 1);
22        else if (op == 2)
23        {
24            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
25        }
26        else if (op == 3)
27            cout << f.sum(p) + 1 << '\n';
28        else if (op == 4)
29            cout << a[f.kth(x) - 1] << '\n';
30        else if (op == 5)
31            cout << a[f.kth(f.sum(p)) - 1] << '\n';
32        else
33        {
34            p = upper_bound(a.begin(), a.end(), x) - a.begin();
35            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
36        }
37    }
38 }

```

算法索引

ac_shortest, 13
ACWeighted, 15
AhoCorasick, 7

Binomial, 76

DC3, 31
DynamicAC, 17

exkmp, 6

Fenwick, 78

GeneralSAM, 46

kmp_match, 1

Lucas, 74

manacher, 61
minimum_rotation, 67
ModInt, 72

OnlineSAM, 53
order_match, 3

PalindromicTree, 64
prefix_function, 1
prefix_lcs, 29

SAIS, 19
sam_lcs, 44
SAMDocuments, 56
SAMLex, 41
square_counts, 29
SuffixArray, 23
SuffixAutomaton, 36
SuffixLCP, 26

z_function, 5